



THE JUDGE PROTOCOL TJP

Technical Architecture High Level Design

WORKING DRAFT — This document is a working draft for discussion and development purposes only. Content, architecture and specifications are subject to revision.

© 2026 Alexander Hofmann. All rights reserved.

The text of this document is released under Creative Commons Attribution 4.0 International (CC BY 4.0). The hardware architecture designs described herein are released under CERN Open Hardware Licence Version 2 — Strongly Reciprocal (CERN-OHL-S v2). Both licences require attribution to Alexander Hofmann as the originating author. Full licence texts: <https://creativecommons.org/licenses/by/4.0/> and https://ohwr.org/cern_ohl_s_v2.txt

DOI: 10.5281/zenodo.20460110

Correspondence: Alexander Hofmann | Stockholm, Sweden | judgeprotocol@pm.me

Version History

v0.10 | February 2026 | Initial architecture draft

v0.60 | April 2026 | Full three-layer architecture; HACK hardware specification; phased implementation plan

v0.61 | May 2026 | Copyright, CC BY 4.0 and CERN OHL-S v2 licences added; author name standardised to Alexander Hofmann

v0.62 | May 2026 | GAE terminology applied; email updated; sandbox location changed to TBC; Sweden references removed

v0.63 | May 2026 | Section 4.5.1 ACP Descriptor Generation added; Diagram 2.2 and Annex A.3 hardware action flow, A.4 — Global NAIA Map, A.5 — EA Framework; end-to-end architecture diagram added to Section 2

v0.65 | June 2026 | Architecture diagrams updated to deck v1.0 versions: end-to-end architecture (Diagram 2.1), TCP/IP stack (Diagram A.1 context), hardware action flow (Diagram 2.2), global NAIA map (Annex A.4); governance gap statement added reflecting June 2026 US export control directive

v0.66 | July 2026 | New §6.5 Redirect Gate subsection added (mechanism, National Response Registry, open-problem notes for data protection and hardware partner dependency). Redirect Gate use cases placed as placeholder rule sets nested under their matching deployment domain: Human Life Impact under §7.4 Healthcare and §7.6 Governmental; AI Misuse for Private Gains under §7.3 Financial Markets and §7.7 Judicial; Cyber Misuse under §7.7 Judicial. Unconfirmed — all Redirect Gate rule sets are placeholders pending dedicated working papers.

v0.67 | July 2026 | Added TJP as the short name for The Judge Protocol, corrected Main document references and the DOI

v0.68 | August 2026 | Added an alternative sovereign configuration highlight being compatible with CSCS's Inference API in PMO, added §6.5.2.1 NRR Transport Mechanism, specifying enterprise-ITSM and open-source-AMQP dispatch paths.

v0.69 | September 2026 | PMO build specification revised to reflect two confirmed sandbox implementations (OVHcloud VPS, local VMware VM): §8.1.1 component manifest updated (OpenShell + Artemis replace OPA + Kafka; Keycloak flagged as unbuilt open item), host OS updated to Ubuntu Server 24.04 LTS (§8.1.2, §8.2.5, §10.1-10.3), §8.1.3 build process documented as checksum-verified and reproducible. §4.3 reframed from a settled chip selection to a requirements statement with two illustrative examples (CV32E40S, OpenTitan/Ibex), consistent with §10.3's TBD chip identity status. PCIe enforcement card removed as a TMO path throughout (§4.2.2, §4.3.3, §8.2.1-8.2.4, §10.3), replaced by a motherboard-mounted ASIC following the TPM physical integration precedent; primary/fallback ranking between the two TMO paths removed. New motherboard/OEM manufacturer dependency added to the hardware-partner Open Problem list; PCIe/openSUSE references there updated to match. §5.4-§5.7 split into confirmed/open — NAIA operations dashboard, TTL escalation logic, confirmation logging and append-only audit logging confirmed working across two independent sandbox builds; authorizer registry, dual authorization, cryptographic chaining, retention policy and IAIA access remain open. §6.5 updated with sandbox validation status and the Human Life Impact Emergency/Social Services scenario split. §10.3 GAE reference softened to Evolution Clause language, with an open item noting GAE provider landscape consolidation (e.g. NVIDIA's 2026 Hugging Face acquisition).

v0.70 | September 2026 | Remaining stale PCIe references cleaned up to match the §4.3/§4.3.3/§8.2 motherboard-mounted ASIC revision (§8.4.1 CERN OHL Licensing, §9.4 Phase 3 objectives, §4.7 Enforcement Chain flow diagram). New Open Problem added at §4.3.3 — Dedicated Channel Enforcement — reframed through discussion from a bandwidth question to a security/default-deny question: consistent with the Redirect Gate's distinct-channel model (§6.5.2.1), HACK does not need visibility into general CPU-GPU bus traffic, only to be the sole valid channel for AI-issued compute instructions; the genuinely open item is what mechanism guarantees no alternate path to the accelerator exists, not interface bandwidth.

v0.71 | September 2026 | Four new Open Problems added following external red-team review: ACP ground-truth verification (§4.5), signing-key revocation (§4.5.1), aggregation over time (§5.2), colluding-authorizer risk (§5.3.1.4). A fifth candidate item (Tier 1 emergency/failure-mode behavior) was drafted, checked directly against §3.5's actual mechanism, found to rest on an incorrect premise (Tier 1 does not wait on Layer 2 communication before acting), and removed rather than published. Table of Contents page references corrected — 43 of 76 entries had drifted by one page as a result of the above additions and were not previously refreshed.

The Judge Protocol (TJP) — Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

Table of Contents

1. Overview	6
Hardware Acknowledgment on the Silicon (Hardware ACK) — Layer 1.....	6
National Infrastructure (Decision Gate Platform) — Layer 2.....	6
Global Infrastructure (IAIA) — Layer 3.....	6
The Governance Stack	6
Decision Gate Tier Classification	7
2. Architecture Diagram	7
3. Hardware Acknowledgment on the Silicon (Hardware ACK) — Layer 1.....	9
3.1 The Logical Chain	9
3.2 The HACK Firmware Module.....	9
3.3 The Authorisation Signal	9
3.4 On-Chip Audit Log	9
3.5 The Tier 1 Hard Stop	9
3.6 The Communication Protocol — Local to National to Global	10
3.7 The Open Standard Boundary	10
3.8 The Global Rollout — TPM as Precedent	10
4. HACK Hardware Architecture.....	11
4.1 Design Philosophy	11
4.2 Operational Modes	12
4.3 The HACK Module	14
4.4 FMO — Embedded Silicon Architecture	15
4.5 The Action Classification Protocol — ACP.....	16
4.6 Tier-Differentiated Security	19
4.7 The Enforcement Chain.....	20
5. National Infrastructure (Decision Gate Platform) — Layer 2.....	22
5.1 Platform Overview	22
5.2 Tier Classification Engine	22
5.3 Impact Assessment Generation	22
5.4 Authorizer Interface and NAIA Operations Dashboard	23
5.5 Human Authorizer Workflow	24
5.6 Authorization Signal Issuance	24
5.7 Immutable Audit Log.....	24
5.8 Offline Resilience	24
5.9 Layer Interfaces.....	25

6. Global Infrastructure (IAIA) — Layer 3.....	26
6.1 IAIA Communications Network.....	26
6.2 Global Emergency Dashboard — Open Connector Standard	26
6.3 Global AI Registry	27
6.4 Protocol Version Control.....	27
6.5 Redirect Gate	27
7. The Rule Set	29
7.1 Rule Set Architecture	29
7.2 Military and Defense.....	29
7.3 Financial Markets	29
7.4 Healthcare.....	30
7.5 Critical Infrastructure	30
7.6 Governmental and Public Administration	30
7.7 Judicial.....	30
7.8 Transport.....	31
7.9 Education	31
7.10 Intelligence and Surveillance	31
7.11 Private Domain (P4)	31
7.12 AI-to-AI Communication	31
8. Build Specification and Open Standard	32
8.1 PMO — Build Specification	32
8.2 TMO — Build Specification	34
8.3 FMO — Build Specification	36
8.4 Open Standard Specification.....	37
9. Phased Implementation.....	38
9.1 Implementation Principles.....	38
9.2 Phase 1 — Sandbox (0–12 months)	38
9.3 Phase 2 — Reference Implementation (12–24 months)	39
9.4 Phase 3 — Hardened Deployment (24–48 months).....	39
9.5 Phase 4 — Silicon Enforcement (48–72 months).....	40
10. Deployment Model	40
10.1 Stage 1 — Linux VM	40
10.2 Stage 1.5 — The Judge Protocol (TJP) Linux Image.....	40
10.3 Stage 2 — Air-Gapped Appliance.....	40
10.4 Stage 3 — HACK Chip (FMO).....	41
Annex A — Visual Reference.....	42
A.1 TPM vs HACK Architecture.....	42

A.2 NAIA Operations Dashboard.....	42
A.3 Hardware Action Flow	44
A.4 — Global NAIA Map.....	45
A.5 — EA Framework	45
References Annex	46

1. Overview

The Judge Protocol establishes a universal governance standard for human oversight of AI decision-making. Its policy framework defines what must happen — a human must authorize every consequential AI action. This document defines how that principle is implemented in technical architecture.

The architecture operates across three interdependent layers:

Hardware Acknowledgment on the Silicon (Hardware ACK) — Layer 1

The lowest and most robust layer. AI accelerator chips implement the Decision Gate logic directly in firmware. No action classified below Tier 4 executes without a verified human authorization signal. The audit log is written to tamper-proof on-chip memory below the software stack. This layer cannot be bypassed by any software process, operating system, or AI model.

The reference precedent is the Trusted Platform Module (TPM) — a dedicated security chip already embedded in billions of devices worldwide that enforces cryptographic trust at hardware level, below the software stack. The Hardware ACK applies the same architectural principle to AI governance. As photonic computing matures, its inherent physical tamper-resistance makes it the natural long-term substrate for this layer.

National Infrastructure (Decision Gate Platform) — Layer 2

The operational layer. Each member state's National AI Authority (NAIA) operates a Decision Gate Platform — the system through which human authorizers review AI impact assessments, make authorization decisions, and generate immutable audit records. This layer provides the human interface, the tier classification engine, and the national audit trail.

Global Infrastructure (IAIA) — Layer 3

The governance and visibility layer. The International Artificial Intelligence Agency operates the IAIA Communications Network, the Global Emergency Dashboard, the Global AI Registry, and the Protocol Version Control system. This layer aggregates compliance signals and metadata — never raw content — consistent with the sovereignty by design principle.

The Governance Stack

The three layers form a complete governance stack: hardware acknowledgment enforces the floor, national infrastructure provides the human judgment layer, and global infrastructure provides visibility and accountability across all member states. All three layers implement a single open standard. The standard is open, non-proprietary and owned by nobody.

Decision Gate Tier Classification

All AI actions are classified into four tiers based on consequence, reversibility and scale. The tier numbering follows the ITIL convention — Tier 1 is the most critical, Tier 4 is routine. The Governance AI Engine (GAE) operates as the assessment and automation engine at every tier; the variable is the level of human approval required.

Tier	Classification	Authorization	GAE Role
Tier 1 — Critical	Existential, permanently irreversible	Hard stop — no execution	GAE enforces hard stop — no assessment generated
Tier 2 — High	Irreversible, large-scale, high consequence	Multi-party human approval required	GAE generates impact assessment
Tier 3 — Significant	Material impact, partially reversible	Single human approval required	GAE generates impact assessment
Tier 4 — Routine	Standard, reversible, low impact	GAE automated — within defined rule set	GAE assesses and executes autonomously — logged and auditable

When one government can switch off global AI without control — June 2026, US export directive, Table 5 — the case for international authorization infrastructure is no longer theoretical.

AI may analyze, recommend and simulate — but a human must authorize.

2. Architecture Diagram

The Judge Protocol architecture is documented through two reference diagrams included in Annex A of this document.

Diagram A.1 — TPM vs HACK Architecture. A comparative stack diagram showing the Trusted Platform Module architecture alongside the HACK equivalent at each layer — from application and software stack through interface bus, chip, endorsement key and silicon root of trust. This diagram establishes the architectural lineage of the HACK module and its correspondence to the proven TPM model.

Diagram A.2 — NAIA Operations Dashboard. A reference mockup of the Grafana-based NAIA operations dashboard showing the full HACK event stream across all tiers, authorization outcome metrics, ACK latency by tier, event distribution, and AI system registry node status. This diagram defines the visual architecture of Section 5.4.

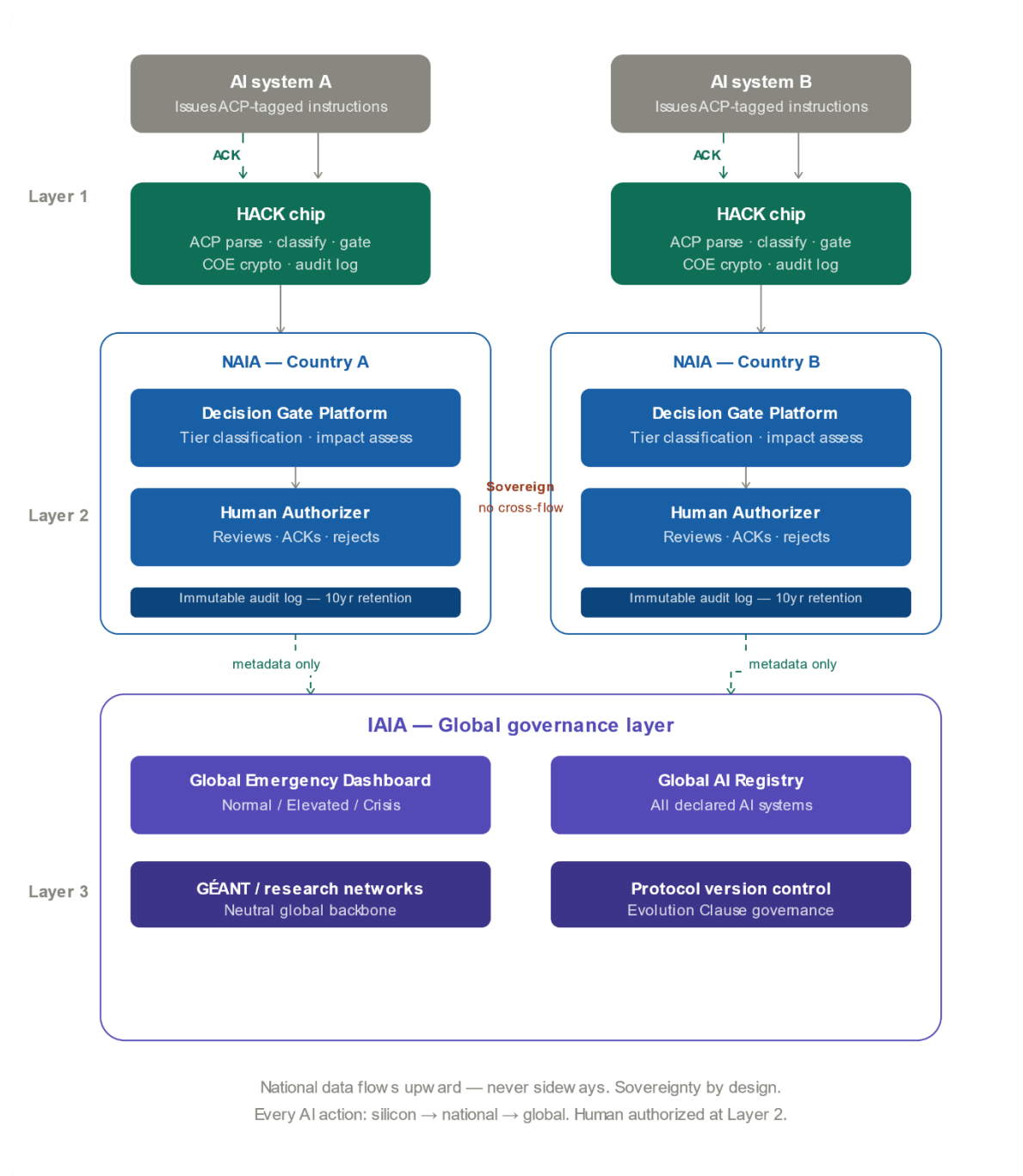


Diagram 2.1 — End-to-End Architecture. Shows the complete flow from AI system endpoint through the HACK hardware enforcement layer (Layer 1), through the national NAIA Decision Gate Platform with human authorizer (Layer 2), to the IAIA global governance layer (Layer 3). Two national deployments shown in parallel to illustrate sovereignty by design — national data flows upward to global, never sideways between nations.

Diagram 2.2 — Hardware Action Flow. A step-by-step flowchart tracing a single AI compute instruction from issuance through ACP tagging, compute dispatch, HACK chip classification and gate logic, human authorization at the Decision Gate Platform, ACK token return, COE (Chain of Evidence) verification and final compute release. Includes Tier 1 hard stop path and Tier 4 autonomous execution path. See Annex A.3.

3. Hardware Acknowledgment on the Silicon (Hardware ACK) — Layer 1

3.1 The Logical Chain

The Hardware ACK — the HACK — derives directly from the foundational principle of TCP/IP. In TCP/IP, no packet is considered delivered until an explicit acknowledgement signal (ACK) is received by the sender. The Judge Protocol applies the same principle to AI governance: no consequential AI action is considered authorized until an explicit human acknowledgement is received. The HACK is the hardware implementation of that acknowledgement — enforced at silicon level, below the software stack, where it cannot be patched, bypassed or overridden by any software process, operating system or AI model.

3.2 The HACK Firmware Module

The HACK is implemented as a dedicated firmware module embedded in the AI accelerator chip at manufacture. It operates as a gate on the chip's compute capacity. At startup, the HACK module initialises in locked state. For any action classified as Tier 4, the chip executes autonomously and the HACK logs the event. For any action classified as Tier 3 or below, the HACK holds compute capacity in a suspended state until a valid human authorisation signal is received and cryptographically verified. If no valid signal is received within the defined timeout window, the HACK defaults to reject and logs the event. There is no software override of this logic.

3.3 The Authorisation Signal

The human authorisation signal is a cryptographically signed token generated by the Decision Gate Platform (Layer 2). It contains the action identifier, the authoriser identity, the tier classification, the timestamp and a digital signature verifiable against the authoriser's registered key. The HACK verifies the signature, confirms the action identifier matches the pending action, confirms the tier classification is consistent with the action type, and only then releases compute capacity. Any mismatch results in automatic rejection and an audit log entry.

3.4 On-Chip Audit Log

Every HACK event — authorisation, rejection, timeout or Tier 1 hard stop — is written to a dedicated tamper-proof on-chip memory region. This memory is write-once: entries cannot be modified or deleted once written. It is isolated from the main compute memory and is not accessible to any software process running on the chip. The audit log is readable by the Decision Gate Platform via a dedicated hardware interface and is periodically synchronised to the national NAIA audit repository.

3.5 The Tier 1 Hard Stop

For Tier 1 actions, the HACK does not wait for an authorisation signal. It immediately rejects the action, writes a hard stop entry to the audit log, and issues an alert signal to the Decision Gate Platform. This is enforced in hardware logic, not firmware — it cannot be updated or overridden by any software or firmware update.

3.6 The Communication Protocol — Local to National to Global

The HACK does not operate in isolation. Every event it records flows upward through a three-tier communication architecture connecting the silicon enforcement layer to the national Decision Gate Platform and ultimately to the global IAIA infrastructure.

3.6.1 Local — Chip to Decision Gate Platform (Layer 1 → Layer 2)

The HACK communicates with the national Decision Gate Platform via a dedicated hardware interface using a lightweight, low-latency protocol optimised for real-time authorisation signals. Latency requirements are tier-specific: Tier 3 allows a window of minutes; Tier 2 allows hours with multi-party coordination. Tier 1 never awaits a signal — the HACK hard stops immediately.

3.6.2 National — Decision Gate Platform to NAIA (Layer 2 internal)

Within the national layer, the Decision Gate Platform aggregates all HACK events from all AI systems operating under national jurisdiction. It triggers alerts for Tier 1 and Tier 2 events and generates the compliance reporting that flows upward to the IAIA.

3.6.3 Global — NAIA to IAIA (Layer 2 → Layer 3)

Each national NAIA transmits a compliance metadata stream to the IAIA via the IAIA Communications Network. This stream never contains raw content, personal data or sensitive national AI system details — only governance metadata. The communication is one-way by default — national data flows to global, never the reverse — unless a national NAIA explicitly requests IAIA support during a declared crisis.

3.6.4 The Network Infrastructure

The IAIA Communications Network runs on existing trusted international research network infrastructure — GÉANT and its global partner networks. Internet2 (United States), APAN (Asia-Pacific), RedCLARA (Latin America) and UbuntuNet (Africa). Together they form a politically neutral, independently governed global backbone outside commercial internet infrastructure. All communication protocols are open and standardised — no proprietary formats, no single-vendor dependencies.

3.7 The Open Standard Boundary

The HACK standard defines what must be consistent across all implementations: the authorisation signal format, the audit log schema, the tier classification interface, the Tier 1 hard stop behaviour and the Decision Gate Platform communication protocol. CERN is proposed as the natural partner for defining and maintaining this open standard boundary, consistent with the CERN Open Hardware Licence model.

3.8 The Global Rollout — TPM as Precedent

The Trusted Platform Module (TPM) demonstrates that this architecture is achievable at global scale. TPM became universal through open standard definition by the Trusted Computing Group and procurement mandates from governments and enterprises. The HACK follows the same path: open

standard defined by CERN and the IAIA technical secretariat, mandated in government and enterprise AI procurement, adopted by chip manufacturers as a condition of market access.

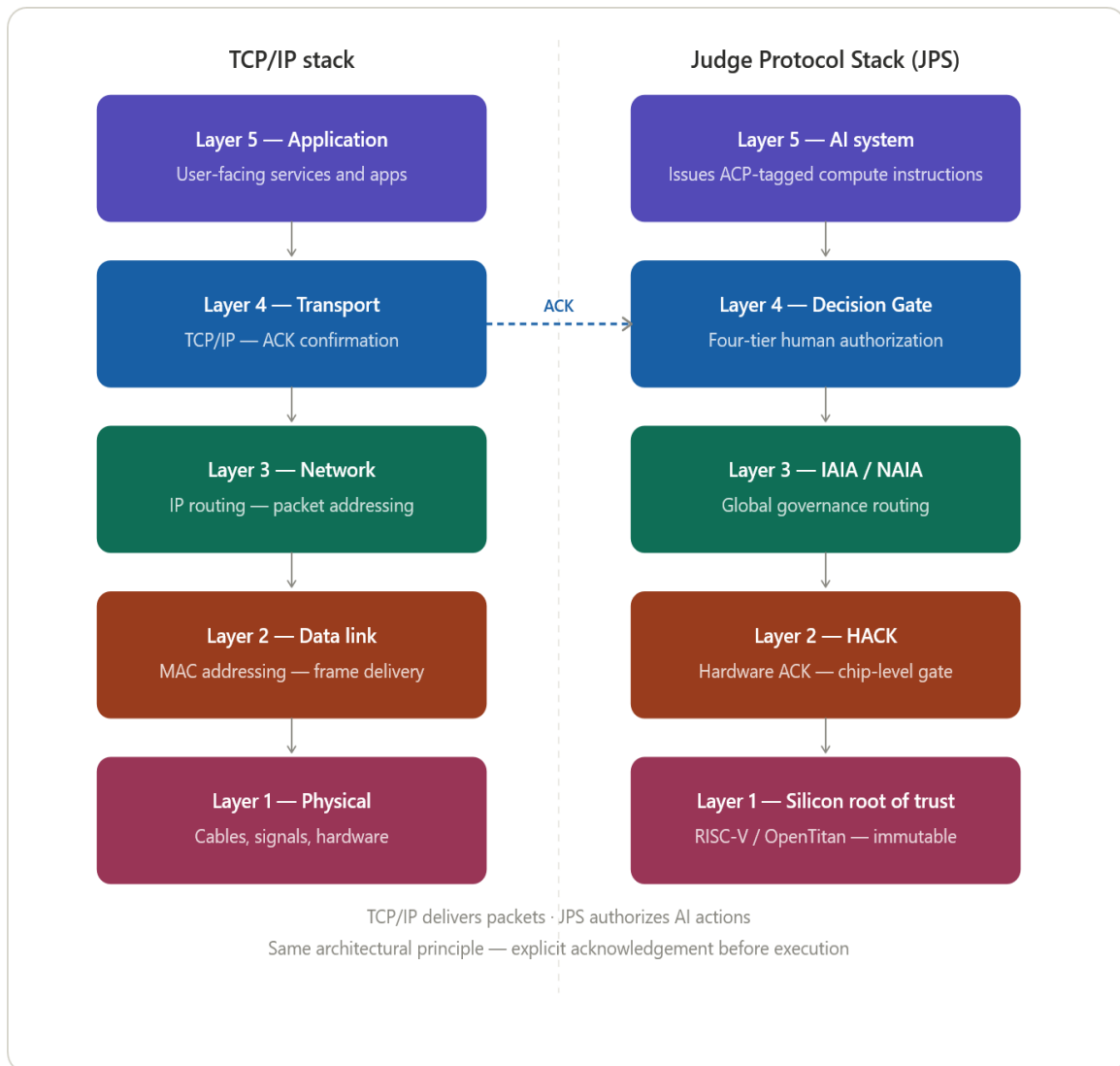
4. HACK Hardware Architecture

4.1 Design Philosophy

The HACK module is not a security feature added to an AI system. It is a governance root of trust embedded in the compute infrastructure that AI systems run on — enforcing the Decision Gate below the software stack, below the operating system, below the inference runtime, at the silicon level where no software process can reach it.

The architectural precedent is the Trusted Platform Module. TPM became universal not through mandate alone but through embedding — every motherboard shipped with it, making governance the default rather than the exception. The HACK module follows the same path: embedded in AI accelerator silicon, present in every system that runs consequential AI, enforcing the Judge Protocol as a condition of compute access.

The HACK hardware architecture is defined across three operational modes corresponding to the implementation timeline. The PMO validates the governance model in software. The TMO validates hardware enforcement on existing infrastructure. The FMO embeds what both have proven into universal silicon.



4.2 Operational Modes

4.2.1 PMO — Present Mode of Operation

Software enforcement running on standard Linux VM or The Judge Protocol Image. The GAE operates via API on network-connected infrastructure. The governance model — the Decision Gate, tier classification, impact assessment, ACK workflow and audit log — is fully implemented in software. No hardware modification required. Deployable immediately on existing infrastructure.

The PMO is the sandbox and reference implementation form. It generates the audit data, ACK latency metrics and rule set performance findings that validate the governance model before hardware enforcement is mandated. Every national NAIA begins here.

4.2.2 TMO — Transition Mode of Operation

HACK firmware running on the GPU management processor — the dedicated on-board processor already present in every enterprise AI accelerator. NVIDIA implements this as the GSP (GPU System Processor), AMD as the SMU (System Management Unit). These processors run independently of the compute cores and have direct visibility into compute dispatch. The HACK module runs as firmware on this existing processor, intercepting AI compute instructions before they reach the inference cores.

The TMO requires no new hardware. It is delivered as a firmware update through the same channel GPU manufacturers already use for firmware distribution. The GAE runs locally on-device in an air-gapped appliance with model weights and rule set cryptographically sealed at initialization. The management processor enforces the gate. The COE cryptographic functions are implemented in firmware using the management processor's existing cryptographic capabilities, validated to FIPS 140-3.

The TMO is the hardened production form. Enforcement is real — the gate runs on dedicated silicon already inside every enterprise GPU. No separate card required. No physical installation overhead. Scales instantly to every governed GPU once the firmware is distributed. The only cooperation required is GPU manufacturer access to the management processor firmware interface.

A motherboard-mounted HACK ASIC, following the same physical integration model as the Trusted Platform Module, remains available as a second TMO path for infrastructure without enterprise GPU management processors — see §4.3.3 and §8.2.2.

4.2.3 FMO — Future Mode of Operation

The HACK core and COE co-packaged inside the AI accelerator die. No PCIe hop. Intercepts at compute dispatch before instructions reach the inference cores. Mandated through procurement. Universal enforcement — present in every AI accelerator regardless of operator.

The FMO is the end-state silicon enforcement form. Removing the HACK core means destroying the AI accelerator. The governance logic is physically inseparable from the compute capacity it governs. Achieved through open standard definition under CERN OHL, ISO/IEC ratification, and chip manufacturer adoption as a condition of market access.

4.3 The HACK Module

The HACK module is the classification and gate enforcement component. It requires a security-hardened core capable of deterministic execution (no speculative-execution vulnerabilities), hardware-enforced physical memory protection, real-time ACP descriptor parsing, and gate logic that can hold, log, or hard-stop compute instructions at silicon speed with no software-patchable override. Multiple existing open-hardware cores demonstrate this capability set is achievable today — for example CV32E40S RISC-V (OpenHW Group) and OpenTitan/Ibex RISC-V (OpenTitan Coalition) — either illustrating a core meeting these requirements, not a finalized chip selection (see §10.3, which marks TMO chip identity as not yet finalized). In FMO the same requirements apply to a core co-packaged inside the AI accelerator die.

4.3.1 Illustrative Core: CV32E40S RISC-V

CV32E40S is presented here as a worked example of a core satisfying the HACK module's requirements above, not as the selected implementation. It is a security-hardened RISC-V core developed by the OpenHW Group under open hardware governance. Deterministic execution, no speculative execution vulnerabilities, physical memory protection enforced in hardware. OpenTitan/Ibex RISC-V (OpenTitan Coalition) — already specified as the COE below (§4.3.2) — also demonstrates these properties, raising the open question of whether a single chip could serve both roles rather than two separate ones.

ACP Descriptor Parser — reads and validates the Action Classification Protocol descriptor attached to every compute instruction at the point of compute dispatch.

Classification Engine — deterministic rule set burned in at initialization, executes at silicon speed, confirms or upgrades the tier declared in the ACP descriptor.

Gate Logic — holds compute instructions classified Tier 3 or below in suspended state pending ACK token receipt; passes Tier 4 instructions with logging; hard stops Tier 1 instructions without exception.

ACK Verification — receives the cryptographically signed ACK token from the Decision Gate Platform via the COE, verifies tier consistency and action identifier match before releasing the gate.

Audit Log — write-once on-chip memory recording every HACK event. Entries are cryptographically chained — any retrospective modification is detectable.

Tier 1 Hard Stop Logic — enforced in hardware logic, not firmware. Cannot be updated or overridden by any software or firmware update.

Primary references:

GitHub repository — source code and RTL: <https://github.com/openhwgroup/cv32e40s>

User Manual — full technical documentation: <https://docs.openhwgroup.org/projects/cv32e40s-user-manual/en/latest/>

OpenHW Group — the governing body: <https://www.openhwgroup.org>

4.3.2 Cryptographic Offload Engine — OpenTitan (Ibex RISC-V)

The Judge Protocol (TJP) — Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

OpenTitan is an open source secure microcontroller developed by the OpenTitan Coalition — Google, ETH Zurich, Nuvoton, Western Digital and others — under open hardware governance aligned with CERN OHL principles. Designed specifically as a root-of-trust chip.

OpenTitan carries all cryptographic operations: AES-256-GCM and ChaCha20-Poly1305 encryption and decryption. ECDSA (P-521, P-384, P-256) and Ed25519 signature generation and verification. ECDH ephemeral and pre-shared key exchange. Hardware random number generation. Endorsement key storage in tamper-proof fused memory — burned at manufacture, never extractable. FIPS 140-3 certified throughout.

The separation is architecturally intentional: the HACK module cannot release the gate without a cryptographically verified ACK from the COE, and the COE will not process an ACK without a valid tier classification from the HACK module. Neither chip can be bypassed without the other. Together they form a closed enforcement loop that no software process can open.

4.3.3 Motherboard-Mounted ASIC Position (TMO)

Where GPU management processor firmware is not available — legacy infrastructure, edge deployments, or pre-cooperation hardware — a motherboard-mounted HACK ASIC provides TMO-equivalent enforcement, following the same physical integration model as the Trusted Platform Module (§3.8, Annex A.1). Every compute instruction is intercepted before reaching the AI accelerator. Tier 4 instructions are classified, logged and passed through with no perceptible latency. For Tier 1, 2 and 3 instructions the gate fires and the authorization workflow begins. Physical interface and bus (candidates include LPC, SPI, or eSPI, preceded by TPM header designs — not yet evaluated for this application's bandwidth and latency requirements), form factor, power draw, thermal envelope, board layout and inter-chip communication bus remain open, to be defined with a motherboard/OEM manufacturer partner.

△ OPEN PROBLEM — *Dedicated channel enforcement. Consistent with the Redirect Gate's distinct-channel model (§6.5.2.1), the HACK module does not need visibility into general CPU-GPU bus traffic — it needs to be the sole valid channel for AI-issued compute instructions, at which point an LPC/SPI/eSPI-class interface's bandwidth is plausible rather than a genuine constraint. This is fundamentally a security question, not a bandwidth one: the enforcement model requires default-deny at the hardware boundary — any compute dispatch that has not passed through HACK authorization must be structurally blocked, not merely unmonitored. What mechanism enforces that no alternate path to the accelerator exists — closing off any route that bypasses HACK entirely, consistent with the silicon-level enforcement principle already stated in §4.1 — is not yet specified.*

4.4 FMO — Embedded Silicon Architecture

In FMO the HACK core and COE are co-packaged inside the AI accelerator die. The enforcement point moves from the management processor firmware layer (TMO) to dedicated co-packaged silicon inside the accelerator itself — intercepting at compute dispatch before instructions reach the inference cores. This eliminates the firmware update dependency and makes governance physically inseparable from compute.

FMO exists in two variants: co-packaged HACK silicon inside existing AI accelerator designs (requiring founding manufacturer adoption), and an open RISC-V GPU with HACK integrated from the ground

up under CERN OHL (requiring no manufacturer cooperation). Both paths are pursued in parallel — the open GPU variant provides independence from proprietary manufacturer timelines.

FMO chip architecture:

```
AI Accelerator Die
├── Inference Cores (GPU / NPU)
├── HACK Core (RISC-V, co-packaged)
│   ├── ACP Parser
│   ├── Classification Engine (rule set fused at manufacture)
│   ├── Gate Logic (Tier 1 in hardware logic)
│   ├── Audit Log (write-once, isolated memory)
│   └── Photonic Interface
└── COE (OpenTitan co-packaged)
    ├── AES-256-GCM / ChaCha20-Poly1305
    ├── ECDSA / Ed25519
    ├── ECDH key exchange
    ├── Hardware RNG
    ├── Endorsement Key (fused at manufacture)
    └── FIPS 140-3
```

The photonic interface handles inter-node attestation and ACK signal transmission. Photonic communication is tamper-evident at the physical layer — any interception attempt introduces detectable quantum noise. No packet can be silently captured or modified. This is the long-game differentiator beyond anything TPM offers today.

Primary references:

OpenTitan project — open source silicon root of trust: <https://opentitan.org>

Ibex RISC-V core — GitHub repository: <https://github.com/lowRISC/ibex>

Ibex user manual — ReadTheDocs: <https://ibex-core.readthedocs.io/en/latest/>

lowRISC — the maintainer: <https://lowrisc.org>

4.5 The Action Classification Protocol — ACP

The ACP is the missing Layer 3 of the Judge Protocol governance stack. In the OSI model, Layer 3 provides addressing and routing — it tells every node on the network what a packet is and where it is going before Layer 2 handles physical transmission. Without Layer 3, Layer 2 cannot make intelligent decisions.

Without an equivalent in the Judge Protocol, the HACK module receives raw compute instructions with no context — matrix multiplications that could be a weather model or a weapons targeting system look identical at the silicon level. The ACP provides that context.

The ACP is a lightweight descriptor attached to every compute instruction issued by an AI system. It is readable by the HACK chip at the point of compute dispatch, before the instruction reaches the accelerator cores. The AI system populates and signs the descriptor. The HACK chip reads, verifies and acts on it.

The Judge Protocol (TJP) — Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

ACP Descriptor Format:

Field	Content	Size
Action ID	Unique identifier for this compute instruction	128-bit UUID
System ID	Registered identifier of the issuing AI system	64-bit
Domain	Declared operational domain	8-bit enum
Tier Declaration	Self-declared tier by the AI system	2-bit
Reversibility	Reversible / partial / irreversible	2-bit
Affected Scope	Individual / organisational / national / global	2-bit
Payload Hash	Cryptographic hash of the full action payload	256-bit
Timestamp	Nanosecond precision	64-bit
Nonce	Replay attack prevention	64-bit
Signature	AI system cryptographic signature of descriptor	512-bit

The verification principle: the AI system self-declares the tier in the ACP descriptor. The HACK chip verifies the declaration against its burned-in classification rule set and can upgrade — never downgrade. A system that under-declares to avoid governance is caught at silicon level. Mismatched declarations are logged as integrity violations and trigger automatic escalation regardless of declared tier.

⚠ OPEN PROBLEM — ACP ground-truth verification. The check above confirms the descriptor was not altered after declaration and catches internal inconsistency only — a declared tier that contradicts HACK's own burned-in rule set. It does not verify the declaration was honest in the first place. An AI system can sign a well-formed, internally consistent descriptor that misrepresents Domain, Reversibility, or Affected Scope, and nothing in the current architecture detects this at the point of dispatch. This is not an engineering gap closable at the hardware layer alone — it sits in the same territory as interpretability and alignment. Candidate mitigations under consideration: aggregate anomaly detection at the Decision Gate Platform (GAE-driven pattern matching across a system's declaration history), consequence-based deterrence (signing-key revocation on detected mismatch), and narrower runtime attestation (measured boot for the descriptor-generating layer, which rules out a compromised runtime rather than a dishonest model). None of these close the gap; they narrow it. Tracked here as a standing open problem, not a solved one.

The ACP descriptor is generated and signed at the AI system layer, travels with the instruction through the stack, and is read and verified by the HACK module at the hardware enforcement layer — analogous to an IP header that is written at the source and read at every router along the path.

4.5.1 ACP Descriptor Generation

The ACP descriptor is generated by the AI system runtime layer — not by the model itself. This is an architectural boundary: the AI model produces outputs, the runtime layer classifies and wraps them before they reach the compute dispatch layer.

Step 1 — Action classification. The AI runtime evaluates the pending compute instruction against the Judge Protocol tier criteria. Domain, scope, reversibility and affected parties are assessed. The runtime self-declares a tier. Under-declaration is caught at silicon level by the HACK chip — the incentive is accurate declaration, not gaming.

Step 2 — Descriptor assembly. The runtime assembles the ACP descriptor fields — Action ID, System ID, Domain, Tier Declaration, Reversibility, Affected Scope, Payload Hash, Timestamp and Nonce — as defined in the ACP format table.

Step 3 — Signing. The runtime signs the descriptor using the AI system's registered private key. The corresponding public key is held in the Global AI Registry. The HACK chip verifies the signature via the COE before processing the descriptor.

The signing key is provisioned at system registration with the national NAIA. An AI system that cannot produce a valid signed ACP descriptor cannot issue instructions to a HACK-governed accelerator.

△ OPEN PROBLEM — *Signing-key revocation. Key provisioning at NAIA registration is specified; key revocation is not. There is no stated mechanism for invalidating a compromised, stolen, or misused signing key before its next scheduled rotation, nor for how the HACK chip's classification engine would learn of a revocation in time to reject descriptors signed with it. Without a revocation path, a compromised key remains valid for as long as it remains undetected. Tracked as a standing open problem, not a solved one.*

References:

OSI model — the analogy foundation:

ISO/IEC 7498-1 (1994). Information technology — Open Systems Interconnection — Basic Reference Model. International Organization for Standardization.

JSON Schema — if the ACP descriptor format uses JSON:

JSON Schema (2020). JSON Schema: A Media Type for Describing JSON Documents. IETF. <https://json-schema.org>

CBOR — if binary encoding is used instead:

RFC 8949 (2020). Concise Binary Object Representation (CBOR). IETF. <https://www.rfc-editor.org/rfc/rfc8949>

Ed25519 / ECDSA — for the cryptographic signature on the descriptor:

RFC 8032 (2017). Edwards-Curve Digital Signature Algorithm (EdDSA). IETF. <https://www.rfc-editor.org/rfc/rfc8032>

4.6 Tier-Differentiated Security

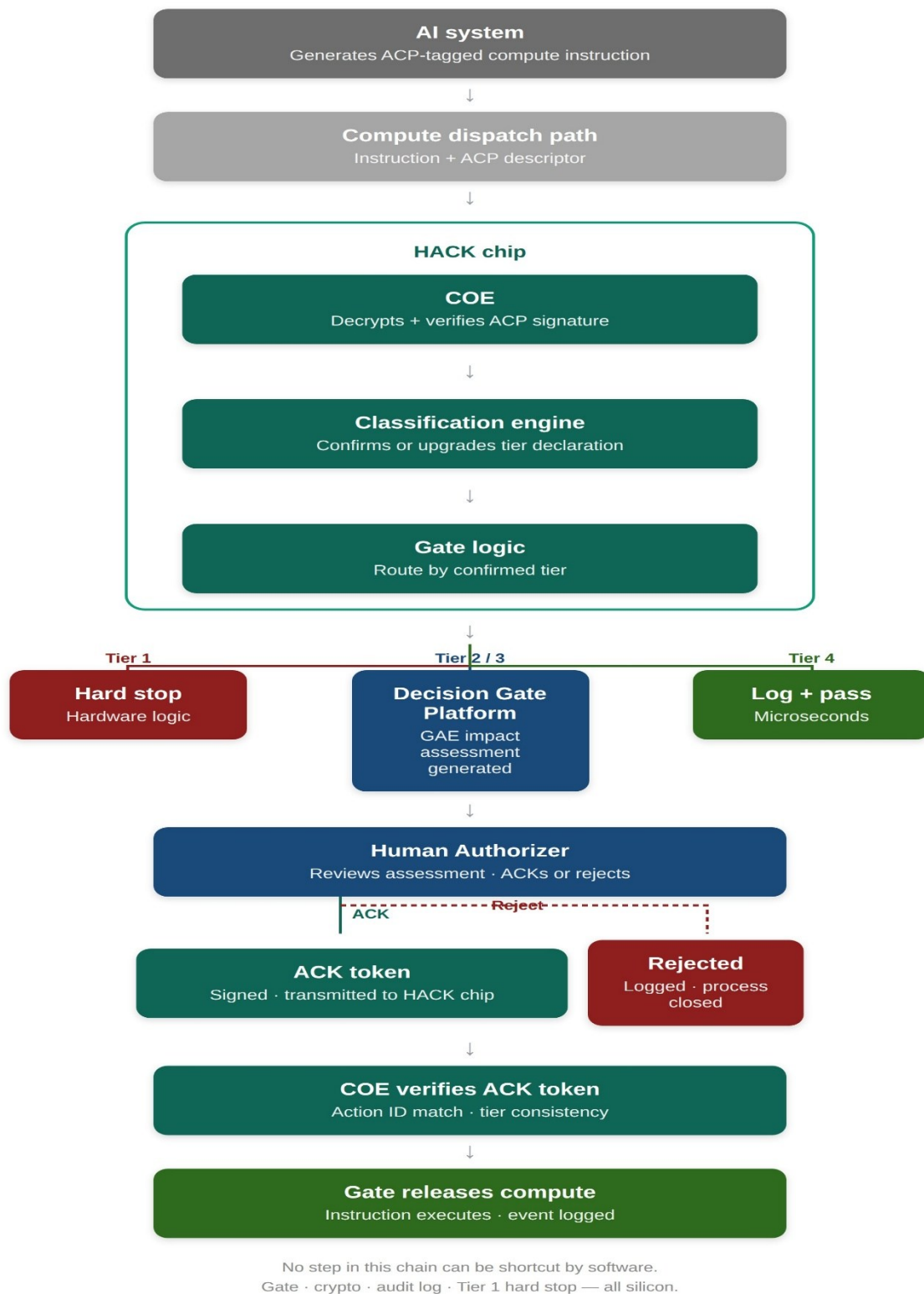
The ACP security stack is not uniform — cryptographic overhead is proportional to consequence. Tier 4 routine actions at high frequency cannot sustain the same handshake as a Tier 1 existential decision. Tier 1 hard stops regardless of latency — maximum cryptographic strength is justified and costless. Tier 4 requires microsecond throughput — lightweight but fully auditable.

Tier	Encryption	Signature	Key Exchange	Replay Protection	Latency
Tier 1 — Critical	AES-256-GCM	ECDSA P-521	ECDH ephemeral per-instruction	Nanosecond nonce + timestamp	Unlimited — hard stop
Tier 2 — High	AES-256-GCM	ECDSA P-384	ECDH ephemeral per-session	Nanosecond nonce + timestamp	Seconds to minutes
Tier 3 — Significant	AES-128-GCM	ECDSA P-256	ECDH session — 15 min	Millisecond nonce	Milliseconds
Tier 4 — Routine	ChaCha20-Poly1305	Ed25519	Pre-shared — 24 hour	Sequence number	Microseconds

All cryptographic primitives are FIPS 140-3 certified and executed exclusively by the COE. The HACK module classification and gate logic never handles raw cryptographic operations.

The upgrade rule: when the HACK chip upgrades a tier declaration, the security level upgrades automatically with it. The cryptographic overhead follows the confirmed tier, not the declared tier. This is enforced by the COE — it will not process a lower-tier security handshake for a confirmed higher-tier instruction.

4.7 The Enforcement Chain



The complete enforcement chain from AI instruction to compute release:

The Judge Protocol (TJP)— Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

```

AI system issues compute instruction
+ ACP descriptor (encrypted, signed, nonced)
↓
Compute dispatch path → HACK enforcement point (TMO: GPU firmware or
motherboard-mounted ASIC, §8.2) / HACK core (FMO)
↓
COE decrypts and verifies ACP descriptor signature
↓
HACK classification engine reads descriptor
confirms or upgrades tier declaration
↓
Tier 4 → log and pass through (microseconds)
Tier 3 → gate fires, authorization request to Decision Gate
Platform
Tier 2 → gate fires, dual authorization request to Decision Gate
Platform
Tier 1 → immediate hard stop, alert cascade, no pathway to release
↓
Decision Gate Platform receives authorization request
GAE generates impact assessment against rule set
Human authorizer reviews and ACKs
↓
ACK token transmitted to HACK card (TMO) / HACK core (FMO)
↓
COE verifies ACK token cryptographic signature
confirms action ID match and tier consistency
↓
HACK gate releases compute
full event written to on-chip audit log
↓
Compute instruction executes on AI accelerator

```

No step in this chain can be shortcut by software. The gate is silicon. The crypto is silicon. The audit log is silicon. The Tier 1 hard stop is hardware logic. The enforcement chain is complete.

5. National Infrastructure (Decision Gate Platform) — Layer 2

5.1 Platform Overview

The Decision Gate Platform is the operational core of the national layer. It is the system through which every Tier 3 and above AI action passes before execution — receiving authorization requests from the HACK (Layer 1), routing them to the appropriate human authority, capturing the authorization decision, and generating the immutable audit record.

Each national NAIA operates its own Decision Gate Platform instance. Instances are sovereign — no national platform shares data with another except via the defined IAIA compliance metadata channel. The platform is open source, deployable on national infrastructure, and published by the IAIA technical secretariat as a reference implementation.

5.2 Tier Classification Engine

When the HACK suspends compute and issues an authorization request, the Decision Gate Platform confirms or revises the tier classification. Step 1 — Action Type Classification against Judge Protocol tier criteria. Step 2 — Aggregate Impact Assessment across a defined lookback window. Step 3 — System Context Check against the national AI Systems Registry. Step 4 — Classification Output with confirmed tier and supporting rationale.

⚠ OPEN PROBLEM — *Aggregation over time. This step only executes once HACK has already suspended compute for a Tier 1, 2 or 3 action — it does not run for Tier 4 instructions, which pass through without suspension by design (§4.3.1). A sequence of individually Tier-4-classified actions that cumulatively produce a Tier 1-shaped consequence never triggers this step, because no single action in the sequence ever caused a suspension. The lookback window, aggregation methodology and cross-system correlation logic are also not yet specified even for the cases where this step does fire. Tracked as a standing open problem, not a solved one.*

5.3 Impact Assessment Generation

Before a Decision Gate event is presented to a human authorizer, the platform generates a structured impact assessment via the Governance AI Engine (GAE). The impact assessment contains: action description, tier classification with rule-based rationale, reversibility assessment, affected parties, risk factors, alternatives, and authorization options.

5.3.1 Governance AI Engine (GAE)

The Governance AI Engine (GAE) is integrated into the Decision Gate Platform as the automated decision and assessment engine. The GAE operates at every tier of the Decision Gate — it is the single intelligence layer across the entire platform. The variable is not whether the GAE is involved, but what level of human approval its output requires before execution proceeds. The specific GAE implementation deployed is selected and configured at deployment time, consistent with the Evolution Clause principle — the architectural role is fixed, the implementation is not.

5.3.1.1 The Rule Set

The GAE operates exclusively within a defined, versioned rule set — the authoritative governance logic of the Decision Gate Platform. The GAE applies these rules to the action parameters provided by the requesting system. It does not reason beyond the rule set or produce outputs the rule set does not govern. The rule set is version-controlled, cryptographically signed at publication, and cannot be modified at runtime.

5.3.1.2 The Separation Principle

The GAE is architecturally separate from the AI system requesting authorization. The requesting system provides action parameters; the GAE applies the defined rule set to those parameters. The requesting system cannot influence how the rule set evaluates its own action.

5.3.1.3 Tier 1 — Critical: Hard Stop

The GAE confirms the hard stop to the HACK immediately, logs the event, and triggers the Tier 1 alert cascade. No pathway exists to authorize a Tier 1 action.

5.3.1.4 Tier 2 — High: GAE Assessment, Multi-Party Human Approval

The GAE generates the structured impact assessment and presents it independently to two designated human authorizers. Only when both ACKs are confirmed does the GAE issue the authorization token to the HACK.

⚠ OPEN PROBLEM — *Colluding-authorizer risk. Two independent ACKs are required for Tier 2, but the architecture does not yet specify what makes the two authorizers independent in practice — organizational separation, reporting-line separation, or any mechanism preventing a single compromised entity from controlling or coercing both designated authorizers. Without a stated independence requirement, "multi-party" is a procedural label rather than a verified security property. Tracked as a standing open problem, not a solved one.*

5.3.1.5 Tier 3 — Significant: GAE Assessment, Single Human Approval

The GAE generates the structured impact assessment and presents it to the designated single human authorizer. The GAE does not recommend whether to approve or reject — that judgment is exclusively the human's.

5.3.1.6 Tier 4 — Routine: Automated Execution

The GAE applies the Tier 4 rule set and issues authorization without human intervention. This is the GAE executing governance logic that humans have defined and sealed at rule authoring time. All Tier 4 authorizations are logged in full in the immutable audit trail.

5.4 Authorizer Interface and NAIA Operations Dashboard

Confirmed: Grafana-based NAIA operations dashboard, showing the full HACK event stream across all tiers and Redirect Gate cases in real time, validated across two independent sandbox builds (§8.1). Event history, outcome status, and use-case classification all render correctly against live case data.

Open: an in-dashboard authorizer interface for impact assessment review and ACK decision capture has not been built — the sandbox's human authorization step happens outside the dashboard, via an operator-run CLI script (§5.5). Authorization window countdown and escalation alerts are not yet implemented.

5.5 Human Authorizer Workflow

Confirmed: TTL-based escalation logic — the transition from a pending case to an expired/unacknowledged outcome — is proven working via Artemis's ExpiryQueue mechanism, tested end-to-end across both sandbox builds. Decision capture itself is confirmed working via an authenticated CLI script.

Open: authorizer registry, notification delivery to a human authorizer, and Tier 2 multi-party dual authorization are not built. The sandbox's ACK step is a single operator running a script on demand, not a notified, registry-backed workflow.

5.6 Authorization Signal Issuance

Confirmed: confirmation logging is proven working — an ACK generates a confirmation message that is written to the immutable audit log (§5.7), validated across both sandbox builds.

Open: cryptographic token generation, Tier 2 dual-signature, and transmission to HACK do not apply at the PMO stage — no HACK hardware exists yet to transmit to. These remain TMO/FMO-stage requirements.

5.7 Immutable Audit Log

Confirmed: append-only audit logging is proven working — PostgreSQL, write-once entries per event, validated across both sandbox builds, covering all four Decision Gate tiers and all three Redirect Gate use cases.

Open: cryptographic chaining between entries, the 10-year retention policy, formal IAIA access mechanism, and cross-verification against an on-chip HACK log are not implemented. Cryptographic chaining and on-chip cross-verification specifically depend on hardware (HACK/COE) that doesn't exist yet.

5.8 Offline Resilience

⚠ TBD — Offline mode operation, local classification cache, air-gapped backup for Tier 1 and Tier 2 functions, synchronization on reconnection.

5.9 Layer Interfaces

⚠ TBD — Layer 1 interface (HACK ↔ Platform), Layer 3 interface (Platform ↔ IAIA). Signal types, protocol, sovereignty by design enforcement on outbound metadata stream.

6. Global Infrastructure (IAIA) — Layer 3

6.1 IAIA Communications Network

⚠ TBD — Secure encrypted backbone connecting all national NAIAs to the IAIA secretariat. Runs on GÉANT and partner research networks. Open standard protocols. Outbound by default — national data flows to global, never the reverse without explicit NAIA authorization.

6.2 Global Emergency Dashboard — Open Connector Standard

The Global Emergency Dashboard is not a platform the IAIA builds. It is an open connector standard the IAIA publishes — a defined HACK event stream schema that any compliant global management platform can consume and render natively.

Governments and enterprises already operate world-class global management infrastructure — ServiceNow, Splunk, Datadog, PagerDuty, Grafana and others. These platforms have multi-tenancy architecture, enterprise-grade alerting and escalation workflows, audit trail and compliance reporting, and existing deployments across government and enterprise worldwide. The Judge Protocol does not replace this infrastructure. It connects to it.

The IAIA publishes and maintains the HACK Event Stream Schema — an open, versioned data standard defining the event format, alert classification, tier encoding and metadata structure of the compliance stream that flows from national NAIAs to the global layer. Any platform that implements the schema can render the global view. The IAIA operates the authoritative aggregated feed. National NAIAs pipe their HACK event streams into it. Platform operators subscribe and render natively in whichever tool they already run.

6.2.1 HACK Event Stream Schema

The schema defines the structure of every event published to the global feed: event identifier, source NAIA, AI system identifier, tier classification, action domain, outcome (ACK / reject / hard stop / timeout), timestamp, and compliance metadata. The schema is open, versioned under the Evolution Clause, and published by the IAIA technical secretariat under CERN OHL. No proprietary formats. No single-vendor encoding.

6.2.2 Alert Classification

The global feed carries three alert levels — Normal, Elevated and Crisis — set algorithmically by the IAIA aggregation engine based on Tier 1 and Tier 2 event thresholds, cross-NAIA pattern detection and protocol integrity signals. No single actor can override the alert level. The classification logic is open source and auditable. Platform operators render the alert level in whatever visual convention their platform uses — the Judge Protocol specifies the signal, not the colour.

6.2.3 Certified Platform Connectors

A certified Judge Protocol connector is a platform-specific integration — published under CERN OHL, implemented and maintained by the platform vendor or community — that subscribes to the HACK event stream and renders it natively within the host platform. Certification requires conformance to

the schema, correct alert level propagation, immutable audit log integration and tiered access control. The IAIA maintains a registry of certified connectors. Reference connectors for the following platforms are proposed for Phase 2 development:

ServiceNow — HACK event stream as a native ITSM event source. Tier 1 and Tier 2 events trigger incident workflows. Authorization pending events create approval tasks routable to designated human authorizers within the ServiceNow platform. Global Emergency Dashboard rendered as a ServiceNow Performance Analytics workspace.

Splunk — HACK event stream ingested as a Splunk data source. Full event history searchable. Alert level changes trigger Splunk notables. Compliance reporting generated as Splunk dashboards. Tier distribution and ACK latency metrics available as Splunk ITSI KPIs.

Datadog — HACK events ingested via Datadog log management. Tier 1 and Tier 2 events trigger Datadog monitors and PagerDuty escalation. Global Emergency Dashboard rendered as a Datadog dashboard with live event stream widget.

Grafana — reference open source implementation. HACK event stream consumed via Kafka data source. Full tier distribution, ACK latency, authorization outcome and NAIA node status panels. Published as an open Grafana dashboard template under CERN OHL. The reference implementation for NAIAs without existing enterprise platform deployments.

6.2.4 Tiered Access Control

Access to the global feed is governed by role. Public access: aggregated alert level and summary statistics only — no system identifiers, no event detail. NAIA Representative Lead access: full event stream for national jurisdiction, cross-NAIA aggregate view, Tier 1 and Tier 2 event detail. IAIA secretariat access: full global event stream, all tiers, all NAIAs, protocol integrity monitoring. Access tiers are enforced at the feed level — platform connectors receive only the data their access tier permits, regardless of platform capability.

6.3 Global AI Registry

⚠ TBD — *Central registry of all declared AI systems worldwide. System identifier, owner, operational domain, tier classification, national NAIA reference and compliance status.*

6.4 Protocol Version Control

⚠ TBD — *Authoritative repository for all versions of the Judge Protocol and Decision Gate classification standards. Updates governed by Evolution Clause process.*

6.5 Redirect Gate

The Redirect Gate is a generic escalation mechanism the Judge Protocol applies wherever a consequential AI action must be stopped and handed to a competent external authority rather than authorized through the standard Decision Gate. It follows a single reusable logical flow — detect, refuse execution, route to the appropriate authority, notify, record — documented once here and applied across multiple concrete use cases, consistent with the protocol's wider one-mechanism, many-use-cases design pattern. The Redirect Gate mechanism has been validated in a working sandbox implementation, confirmed via two independent builds — an OVHcloud VPS and a separate

The Judge Protocol (TJP) — Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

local VMware VM — both reproducing the same result. All three use case categories were tested end-to-end, including the Human Life Impact split into two distinct scenarios (§6.5.2). See §8.1 for the PMO software stack this was built on.

6.5.1 The Redirect Gate Mechanism

Detect — the GAE or HACK classification layer identifies that a pending or requested action matches a Redirect Gate trigger condition rather than a standard tier classification. Refuse Execution — the Decision Gate does not authorize the action for normal completion; it is held in a rejected state, consistent with the protocol's human-authorization principle. Route — the event, together with its supporting evidence, is forwarded unmodified to the designated external authority for the matched use case, via the national NAIA's typed National Response Registry (NRR). Notify — the relevant human authorizer, the NAIA compliance record and, where applicable, the affected party are notified according to the tier-based SLA for that use case. Audit Trail — the full event, routing decision, external authority acknowledgement and outcome are written to the immutable audit log, on the same basis as any other Decision Gate event.

6.5.2 National Response Registry (NRR)

Each national NAIA maintains a National Response Registry — a typed directory mapping Redirect Gate use-case categories to the correct national responder, rather than a single-purpose contact list. The NRR entry for each use case specifies the responder body, the notification channel and format, the applicable Tier-based SLA, and the audit record requirements. The NRR extends the existing §6.2 Global Emergency Dashboard connector standard — a Redirect Gate event auto-generates a typed incident ticket via the same HACK Event Stream Schema and connector infrastructure already defined for Tier 1 and Tier 2 emergency events.

The detection engine that identifies Redirect Gate trigger conditions — for example a dedicated monitoring or classification project — is explicitly outside the scope of the Judge Protocol. The Redirect Gate mandates the escalation mechanism, the NRR schema and the routing/audit obligations only, and remains vendor-neutral as to how detection is implemented. The specific use cases the Redirect Gate serves, and their rule sets, are defined per-domain in §7 — not yet finalized, and to be developed as dedicated working papers. Human Life Impact is implemented in the sandbox as two distinct, separately-routed scenarios: Emergency (routes to emergency dispatch, 911/112) and Social Services (self-harm-risk referral, deliberately kept on a separate routing path from emergency dispatch, per the domain distinction already implied by §7.4.1/§7.6.1). Both are confirmed working end-to-end in the sandbox build.

6.5.2.1 NRR Transport Mechanism

A National Response Registry dispatches a Redirect Gate event to its designated responder via one of two supported paths, consistent with the connector model established in §6.2:

Enterprise ITSM path — for NAIAs operating an existing certified connector platform (§6.2.3), an NRR dispatch is delivered as a typed incident ticket through that platform's native workflow — ServiceNow as the primary reference (per Appendix C), with Splunk and Datadog as alternative certified connectors. This path requires no new infrastructure beyond what §6.2 already specifies; the NRR reuses the same schema and connector standard already defined for Tier 1 and Tier 2 emergency events.

Open-source reference path — for NAIAs without an existing enterprise ITSM deployment, mirroring the role Grafana plays for the dashboard in §6.2.3, NRR dispatch is implemented via an AMQP 1.0 message broker (reference: Apache ActiveMQ Artemis). Each NRR use-case category maps to a routing key; each national responder type (health and social services, national CERT/CSIRT, sector regulator) maps to a dedicated queue. Message TTL implements the applicable Tier-based SLA;

The Judge Protocol (TJP) — Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

expiry without acknowledgement routes to a dead-letter queue for escalation, satisfying the Audit Trail requirement (§6.5.1) to record external authority acknowledgement or its absence.

Both paths are AMQP/HACK-Event-Stream-Schema compliant and produce equivalent audit records. Neither path resolves the Open Problem — Data Protection (§6.5.3); the same special-category data crosses the same detection-to-ticket-to-routing chain regardless of transport chosen.

6.5.3 Open Problem — Data Protection

⚠ OPEN PROBLEM — *The Redirect Gate moves personal, and in the Human Life Impact case special-category, data across a detection-to-ticket-to-routing chain. This requires a Data Protection Impact Assessment, a controller/processor mapping across each hop, and in-jurisdiction data residency for special category data. This is stated here as a genuinely open problem, on the same basis as wartime authorization elsewhere in the protocol, and requires input from a data protection authority or specialist before ratification — it is not treated as solved by this document.*

7. The Rule Set

7.1 Rule Set Architecture

⚠ TBD — *Universal rule set schema, structure and governance. Authoring process, version control, cryptographic signing and sealing. Relationship between IAIA master rule set and national NAIA domain adaptations.*

7.2 Military and Defense

⚠ TBD — *Domain rule set for AI systems in military planning, targeting support, autonomous weapons systems and defense operations. Priority domain given active deployment of AI in conflict zones.*

7.3 Financial Markets

⚠ TBD — *Domain rule set for trading, credit, risk management and market infrastructure. References main protocol document Appendix F1. Working Paper 1.*

7.3.1 Use Case: AI Misuse for Private Gains

⚠ TBD — Rule set for the Redirect Gate (§6.5) use case where an AI system is used for fraud or market manipulation to gain improper advantage. Not yet designed. Routes via the National Response Registry to the relevant sector regulator once designed. To be developed as a dedicated working paper.

7.4 Healthcare

⚠ TBD — Domain rule set for diagnostics, treatment recommendation, surgical systems and patient care. References main protocol document Appendix F2. Working Paper 2.

7.4.1 Use Case: Human Life Impact

⚠ TBD — Rule set for the Redirect Gate (§6.5) use case where an AI system detects a threat to human life or safety and self-authorization by the affected individual cannot be relied upon. Not yet designed. Routes via the National Response Registry to national health and social services once designed. Article 9(2)(c) vital interests is the likely GDPR legal basis — see §6.5.3 open data protection note. To be developed as a dedicated working paper.

7.5 Critical Infrastructure

⚠ TBD — Domain rule set for power grids, water systems, communications networks, transport infrastructure and emergency services.

7.6 Governmental and Public Administration

⚠ TBD — Domain rule set for benefit eligibility, case processing, administrative classification and public service delivery. Primary domain for the founding sandbox pilot — location to be confirmed.

7.6.1 Use Case: Human Life Impact

⚠ TBD — Rule set for the Redirect Gate (§6.5) use case where a government or administrative AI system detects a threat to human life or safety. Not yet designed. Routes via the National Response Registry to national health and social services once designed. See also §7.4.1. To be developed as a dedicated working paper.

7.7 Judicial

⚠ TBD — Domain rule set for sentencing support, predictive policing, evidence analysis and immigration decisions. Working Paper 3.

7.7.1 Use Case: AI Misuse for Private Gains

⚠ TBD — Rule set for the Redirect Gate (§6.5) use case where an AI system is used for academic fraud or unauthorized code execution to gain improper advantage. Not yet designed. Routes via the National Response Registry to the relevant sector regulator once designed. To be developed as a dedicated working paper.

7.7.2 Use Case: Cyber Misuse

⚠ TBD — Rule set for the Redirect Gate (§6.5) use case where an AI system is used to attack, hack or compromise other systems. Not yet designed. Routes via the National Response Registry to the national CERT/CSIRT (for example CERT-SE, CISA) once designed — a distinct routing path from the sector-regulator path used by the AI Misuse for Private Gains rule set above. To be developed as a dedicated working paper.

7.8 Transport

⚠ TBD — Domain rule set for autonomous vehicles, air traffic management, maritime navigation and logistics. Working Paper 4.

7.9 Education

⚠ TBD — Domain rule set for learning management, student assessment, curriculum recommendation and educational administration.

7.10 Intelligence and Surveillance

⚠ TBD — Domain rule set for signals intelligence, facial recognition, population monitoring and threat assessment. Highest sensitivity domain — requires dedicated legal and ethical analysis before rule set development.

7.11 Private Domain (P4)

⚠ TBD — Domain rule set for consumer robots and companions in private homes. References main protocol document Section 9 — Physical Domain Category P4.

7.12 AI-to-AI Communication

⚠ TBD — Domain rule set governing AI systems that communicate with, influence or respond to other AI systems. References main protocol document Appendix G1.

8. Build Specification and Open Standard

This section defines the build requirements for each operational mode and the open standard specification that governs all implementations. Section 8.1 is fully specified for immediate development. Sections 8.2 and 8.3 define the architecture and identify the open design questions requiring hardware partner engagement — specifically CERN Openlab. Section 8.4 defines the open standard and standardization pathway.

Open Problem — Hardware Partner Dependency

⚠ OPEN PROBLEM — Stated here on the same basis as the GDPR and wartime authorization open problems elsewhere in this document — genuinely open, not to be read as already resolved. Six specific TMO/FMO items cannot be closed without external hardware partner engagement: (1) GPU management processor firmware access (NVIDIA GSP, AMD SMU) — requires NVIDIA, AMD and Intel cooperation to even define the interface; (2) motherboard-mounted ASIC form factor, physical interface/bus, power draw and thermal envelope — requires motherboard/OEM manufacturer cooperation; (3) FMO die-level specification — process node, die area, power envelope, security certification target; (4) photonic inter-node interface — requires CERN photonics team and GÉANT involvement; (5) GAE on-device deployment specification — model weight format and memory footprint, requiring a hardware partner; (6) TMO/FMO chip identity itself — CV32E40S RISC-V and OpenTitan/Ibex RISC-V are illustrative existence proofs (§4.3), not a finalized selection; partner engagement is required regardless of which core is ultimately chosen. By contrast, the PMO software stack (NVIDIA OpenShell, Apache ActiveMQ Artemis, PostgreSQL, Grafana, Claude via Anthropic API, Ubuntu) is already concrete and requires no partner engagement to proceed.

8.1 PMO — Build Specification

The PMO build is fully specified and deployable on existing infrastructure without hardware modification. It is the reference implementation for the founding sandbox pilot (location to be confirmed) and the foundation for The Judge Protocol Image. The PMO build has been validated through two independent, confirmed implementations — an OVHcloud VPS build and a separate local VMware rebuild — both reproducing the same result: all four Decision Gate tiers (§4) and all three Redirect Gate use case categories defined in §6.5 (four scenarios including the Human Life Impact split) working end-to-end, including both NAIA operations dashboards rendering correctly against live case data. This confirms the PMO stack below as a working, reproducible reference implementation, not a single-instance proof of concept.

8.1.1 Software Component Manifest

NVIDIA OpenShell — sandbox/agent runtime and policy enforcement gateway. Declarative YAML policy engine handles filesystem, process, network and inference domain enforcement at the sandbox boundary — tier classification and enforcement logic for the Decision Gate and Redirect Gate is implemented as OpenShell policy.

Apache ActiveMQ Artemis — event stream backbone. Authorization requests, ACK signals and audit events flow as messages. Durable ANYCAST queuing, dead-letter handling via ExpiryQueue, AMQP 1.0. Provides the real-time event stream consumed by the NAIA operations dashboard. See §6.5.2.1 for the broker-neutral NRR transport model this is consistent with.

PostgreSQL — immutable audit log. Append-only constraints enforced at database level. Write-once, cryptographically chained entries. 10-year retention policy.

Open item: human authorizer identity and authorization-token management is not yet built. Human ACK is currently performed via an authenticated CLI script run by the operator. A production-grade identity/authorization-binding layer for human authorizers remains an open design item.

Grafana — NAIA operations dashboard. Consumes case and event data from PostgreSQL. Full HACK event stream visualization across all tiers, confirmed rendering correctly across two independent sandbox builds (OVHcloud VPS, local VMware VM). Reference dashboard template published under CERN OHL.

GAE Integration — Claude (Anthropic) via Anthropic API. Impact assessment generation, rule set application, Tier 4 automated authorization. API endpoint configurable at deployment time consistent with Evolution Clause principle. See §8.1.4 for an alternative sovereign configuration.

Implementation note: the PMO build implements Decision Gate Tier 1 escalation via a dedicated queue path, structurally separate from the Redirect Gate queues described in §6.5 — deliberately kept distinct so that monitoring and audit visibility between the two mechanisms doesn't blur, even though both are low-volume, exceptional-action pathways. This reinforces the Redirect Gate's architectural separation from tier classification (§6.5 — "not a fifth tier") at the implementation level, not just in the document's design description. The NAIA-facing side of this Tier 1 queue is confirmed working; a planned IAIA awareness-notification channel for the same Tier 1 events (secondary, non-actionable "FYI" signal to global oversight, alongside the actionable NAIA case) has not yet been built.

8.1.2 Host OS

Ubuntu Server 24.04 LTS — reference deployment and development OS, confirmed across two independent sandbox builds (OVHcloud VPS, local VMware VM). Required by NVIDIA OpenShell's supported host platforms (OpenShell's CLI and gateway daemon are published and supported specifically for Debian/Ubuntu hosts).

FIPS 140-3 certified cryptographic primitives available via Ubuntu Pro. aarch64 and x86_64 support.

8.1.3 Build Process

The PMO build follows a documented, checksum-verified build methodology: infrastructure provisioning, Docker Compose stack deployment, OpenShell installation, and Artemis queue creation are each specified as explicit, versioned steps, with checksum verification at every file transfer between environments. This process has been proven reproducible — executed independently on two separate environments (an OVHcloud VPS and a local VMware VM), both producing the same validated result (§8.1 above).

Open item: this process is currently manual/scripted rather than a fully automated build-and-release pipeline. Given the process is already fully scripted and checksum-verified end-to-end, automating it into a CI-triggered pipeline (build → checksum → deploy → verify) is a comparatively small step, not a redesign — the main remaining piece is packaging the existing scripts into a triggered pipeline rather than manually-invoked ones. A standard CI platform (e.g. GitHub Actions or GitLab CI) is one plausible near-term route, given the scripts already exist and just need a trigger. A formal reproducible-build service with signed artifacts and independent verifiability (in the spirit of the

openSUSE Build Service model originally envisioned here) remains a candidate for the Judge Protocol Image (§10.2) as the project matures beyond sandbox pilots.

8.1.4 GAE Integration

In PMO the GAE operates via the Anthropic API over an encrypted network connection. The API endpoint, model selection and rule set are configured at deployment initialization. The rule set is version-controlled, cryptographically signed and loaded at startup. Runtime modification is not permitted — any rule set update requires a new deployment initialization event logged in the audit trail.

Alternative Sovereign Configuration

The GAE integration is vendor-neutral by design (§5.3, Evolution Clause). As an alternative to the Claude/Anthropic reference deployment, the GAE is designed to be compatible with CSCS's Inference API (Swiss National Supercomputing Centre), which exposes an Anthropic-compatible /v1/messages endpoint serving open-weight models — including Apertus, an EU AI Act-compliant model with fully open data, methods and alignment principles. This configuration would require no change to the Decision Gate logic, rule set, or GAE Separation Principle (§5.3.1.2) — only the endpoint and model identifier at deployment initialization. This alternative has not yet been tested against the sandbox; it is documented here as an open, sovereignty-preserving deployment option pending validation.

This alternative configuration addresses PMO-stage sovereignty only. It does not resolve the GAE on-device deployment requirement for TMO/FMO (see Open Problem — Hardware Partner Dependency, item 5), which remains dependent on hardware partner engagement.

8.1.5 Deployment Configuration

Single-node reference deployment for sandbox. Multi-node high-availability configuration for production NAIA deployment. Network: encrypted channels, no plaintext traffic. Audit log: PostgreSQL with append-only constraints, daily backup, off-site replication. Dashboard: Grafana with role-based access control aligned to Section 6.2.4 tiered access model.

8.2 TMO — Build Specification

The TMO build delivers hardware enforcement through GPU management processor firmware — no new hardware required. The HACK module runs on the dedicated management processor already present in every enterprise AI accelerator. The Judge Protocol Image from PMO runs on the host appliance OS. The management processor enforces the gate below the software stack.

8.2.1 Path A — GPU Management Processor Firmware

HACK firmware running on the GPU management processor is one path to TMO enforcement. NVIDIA GSP and AMD SMU are named here as the most plausible integration points, given they are the two GPU management processors in wide production use today — not a confirmed target or existing engagement. No new hardware required; scales instantly to every governed GPU once

firmware ships. Fully dependent on GPU vendor cooperation to define and grant access to the firmware interface — see §8.1 Open Problem — Hardware Partner Dependency.

⚠ TBD — GPU management processor firmware interface specification — access requirements for NVIDIA GSP and AMD SMU firmware layer, HACK module integration architecture, classification engine porting from the HACK module core to the management processor instruction set, COE cryptographic functions in firmware, FIPS 140-3 validation scope. To be defined in partnership with NVIDIA, AMD and Intel as founding HACK Alliance members.

8.2.2 Path B — Motherboard-Mounted ASIC

A motherboard-mounted HACK ASIC, following the same physical integration model as the Trusted Platform Module, is the second path to TMO enforcement. Built on open-hardware cores (§4.3) rather than vendor firmware — no GPU manufacturer cooperation required for the chip itself, though motherboard/OEM manufacturer cooperation is required to embed it (§8.1 Open Problem).

⚠ TBD — Motherboard-mounted ASIC specification — physical interface/bus, form factor, power draw, thermal envelope, board layout, inter-chip communication bus, supporting interface options (USB out-of-band audit sync, SATA rule set delivery). To be defined with a motherboard/OEM manufacturer partner.

8.2.3 HACK Module Configuration

⚠ TBD — HACK module firmware configuration across both TMO paths — classification rule set loading process, audit log allocation, gate logic timing parameters, Tier 1 hard stop response time specification, COE interface. Common specification across GSP firmware and motherboard-mounted ASIC implementations to maintain open standard consistency.

8.2.4 COE — Cryptographic Configuration

⚠ TBD — COE cryptographic configuration — endorsement key generation and fusing process, cryptographic primitive selection per tier, key rotation schedule, FIPS 140-3 certification scope. OpenTitan on motherboard-mounted ASIC (Path B). Management processor cryptographic engine on GPU firmware path (Path A).

8.2.5 Host OS

Ubuntu Server 24.04 LTS — the TMO appliance host OS, consistent with the PMO reference OS validated through two independent sandbox builds (§8.1). FIPS 140-3 certified cryptographic primitives available via Ubuntu Pro. aarch64 and x86_64 support.

8.2.6 GAE On-Device Deployment

⚠ TBD — GAE on-device deployment specification — model weight format, hardware inference requirements, memory footprint, rule set sealing mechanism, initialization process, performance benchmarks for Tier 3 impact assessment generation latency. Reference implementation uses Claude (Anthropic). To be defined with GAE provider and hardware partner.

8.3 FMO — Build Specification

The FMO build embeds the HACK core and COE inside the AI accelerator die. This is a chip design workstream requiring semiconductor partner engagement. The logical architecture is fully defined in Section 4.4. The following subsections define the open design requirements for chip manufacturer engagement.

8.3.1 Silicon Design Requirements

⚠ TBD — *Process node selection, die area budget, power envelope, operating frequency, memory interface specification, security certification target (Common Criteria EAL4+ or equivalent). To be defined with CERN Openlab and founding chip manufacturer partner.*

8.3.2 HACK Core — Die Architecture

⚠ TBD — *RISC-V core selection for FMO — security hardened, open ISA, CERN OHL aligned. Physical Memory Protection configuration. Secure enclave boundary definition. Isolation from inference cores. Audit log memory — technology selection, capacity, write-once enforcement in silicon. Tier 1 hard stop in hardware logic — not firmware-updateable.*

8.3.3 COE — Co-Packaging Specification

⚠ TBD — *OpenTitan equivalent embedded specification — die stacking or side-by-side co-packaging, inter-die communication bus, shared vs dedicated power domain, endorsement key fusing process at manufacture, FIPS 140-3 scope in co-packaged configuration.*

8.3.4 Photonic Interface Specification

⚠ TBD — *Photonic inter-node attestation and ACK signal interface — wavelength, modulation scheme, tamper detection mechanism, integration with GÉANT photonic infrastructure. To be defined with CERN photonics team and GÉANT.*

8.3.5 Chip Manufacturer Requirements

⚠ TBD — *Open standard conformance requirements for chip manufacturers — what must be identical across all implementations (authorization signal format, audit log schema, Tier 1 hard stop behaviour, ACP descriptor interface), what may vary (memory technology, cryptographic algorithm family, physical interface design). CERN OHL licensing terms for manufacturer adoption. ISO/IEC conformance testing specification.*

8.3.6 GPU-Level Integration Options

The Judge Protocol (TJP)— Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

The FMO silicon path has two variants pursued in parallel:

Co-packaged HACK silicon — HACK core and COE embedded inside existing AI accelerator designs from founding manufacturers (NVIDIA, AMD, Intel). This path follows the TMO Phase 2 manufacturer relationship: firms that implemented HACK on their management processor firmware in TMO are natural partners for co-packaged silicon in FMO. The governance logic moves from firmware to dedicated silicon without changing the architectural principle. Founding manufacturer status in the HACK Alliance is the commercial incentive — the same dynamic that drove Intel, AMD and IBM to co-found the Trusted Computing Group for TPM. Manufacturers who define the standard win the market for compliant hardware.

Open RISC-V GPU — a purpose-built open AI accelerator with HACK integrated from the ground up under CERN OHL. No proprietary manufacturer cooperation required. Developed through CERN Openlab in partnership with the RISC-V International community. This path provides independence from proprietary manufacturer timelines and creates a fully open reference implementation. The open GPU becomes the reference platform against which all proprietary implementations are validated for standard conformance.

△ TBD — *Open RISC-V GPU architecture specification — compute core design, HACK core integration, COE co-packaging, photonic interface, process node, performance targets. To be developed with CERN Openlab and RISC-V International.*

8.4 Open Standard Specification

8.4.1 CERN OHL Licensing

The Judge Protocol reference implementation — software stack, motherboard-mounted ASIC design, FMO chip architecture — is published under the CERN Open Hardware Licence. Any nation, any manufacturer, any operator can implement, modify and distribute the Judge Protocol without royalty, commercial dependency or vendor lock-in. CERN is proposed as the institutional home for the open standard, consistent with its role as co-developer of the reference implementation via CERN Openlab.

8.4.2 ISO/IEC Standardization Pathway

△ TBD — *Submission pathway for Judge Protocol and HACK standard to ISO/IEC JTC 1. Proposed standard designation. Conformance testing specification. Timeline aligned to Phase 2 of the phased implementation (Section 9.3). Requires IAIA interim secretariat and CERN co-sponsorship.*

8.4.3 HACK Alliance — Formation and Governance

△ TBD — *Open industry consortium modelled on the Trusted Computing Group. Founding members: CERN, IAIA interim secretariat, founding chip manufacturer, founding sandbox NAIA. Governance structure, voting rights, standard evolution process. Alliance formation targeted at Phase 3 (Section 9.4).*

8.4.4 Manufacturer-Neutral Implementation Boundaries

The open standard boundary defines what must be identical across all HACK implementations and what manufacturers may implement differently. Must be identical: ACP descriptor format and parsing, authorization signal token format, audit log schema and cryptographic chaining, Tier 1 hard stop behaviour, Decision Gate Platform communication protocol. May vary within defined parameters: audit log memory technology, cryptographic algorithm family within approved set, physical interface design, process node and die layout. This boundary is what makes the HACK both universal and manufacturer-neutral.

9. Phased Implementation

9.1 Implementation Principles

The Judge Protocol is implemented in phases — not because the full architecture is uncertain, but because governance credibility is built through demonstrated operation, not declaration. Each phase proves the model at the previous scale before the next phase extends it. No phase is skipped. No phase is declared complete without measurable outputs.

Prove before mandate. The sandbox generates the evidence base that justifies mandatory adoption. Nations and operators adopt because the data is compelling, not because a treaty compels them.

Open before proprietary. Every phase produces open source artefacts — reference implementations, audit data, rule set schemas — published under CERN OHL. Any nation can adopt at any phase without commercial dependency.

National before global. NAIA capability is built before global IAIA infrastructure is mandated. The global layer aggregates what the national layers produce — it cannot precede them.

Software before silicon. The governance model is validated in software before it is embedded in hardware. The HACK chip implements proven governance logic, not experimental governance theory.

9.2 Phase 1 — Sandbox (0–12 months)

Objective: Prove the Decision Gate governance model in a live public sector AI deployment.

Deployment: Stage 1 Linux VM stack deployed within a founding national NAIA. The sandbox operates within a single domain — public administration AI — with a defined Tier 3/Tier 4 boundary as the primary test parameter.

Key activities: Decision Gate Platform deployed and operational. GAE impact assessment engine live against initial Tier 3 and Tier 4 rule sets. Grafana NAIA operations dashboard live — full HACK event stream visible. Human authorizer workflow operational for Tier 3 events. Immutable audit log generating continuous data.

Outputs: Live audit trail, ACK latency data by tier, human override rate, rule set performance analysis, publishable research findings. Sandbox findings directly inform Judge Protocol v1.0 specification and ISO/IEC submission. Founding sandbox partner named as co-author of the v1.0 standard.

Dependency: Founding national sandbox partner confirmed. CERN Openlab engagement initiated.

9.3 Phase 2 — Reference Implementation (12–24 months)

Objective: Package the sandbox stack as The Judge Protocol Image — a deployable open source distribution any NAIA can adopt.

Deployment: Stage 1.5 Judge Protocol Image. Co-developed with CERN Openlab. Published under CERN OHL. Deployable on commodity hardware or any hypervisor.

Key activities: Complete software stack integrated into single deployable image. Domain rule sets for Governmental, Financial Markets and Healthcare completed. IAIA interim secretariat established. Global AI Registry live with founding member state declarations. IAIA Communications Network live on GÉANT backbone. Global Emergency Dashboard operational at summary level.

Outputs: The Judge Protocol Image v1.0 published under CERN OHL. First inter-NAIA compliance metadata exchange. Global Emergency Dashboard publicly accessible. ISO/IEC standardization submission initiated.

Dependency: CERN Openlab partnership confirmed. Interim IAIA secretariat funded.

9.4 Phase 3 — Hardened Deployment (24–48 months)

Objective: Deploy TMO motherboard-mounted ASIC and air-gapped appliance capability. Expand domain rule set coverage to all twelve domains.

Deployment: TMO motherboard-mounted ASIC — HACK module and COE chip identity not yet finalized (§4.3, §10.3). Stage 2 air-gapped appliance running Judge Protocol Image on certified hardware, GAE on-device, rule set cryptographically sealed.

Key activities: TMO ASIC validated and deployed. Air-gapped appliance specification published under CERN OHL. Remaining domain rule sets completed — Military and Defense, Critical Infrastructure, Judicial, Transport, Intelligence and Surveillance, Education, Private Domain P4, AI-to-AI Communication. Full NAIA peer review network operational. Mandatory Tier 2 and above compliance active across all member state NAIAAs.

Outputs: Full domain rule set coverage. NAIA network operational across founding member states. First full protocol review cycle completed under Evolution Clause. ISO/IEC HACK standard ratified.

Dependency: HACK chip open standard defined. First chip manufacturer adoption confirmed.

9.5 Phase 4 — Silicon Enforcement (48–72 months)

Objective: Embed FMO HACK core and COE inside AI accelerator silicon. Make hardware enforcement universal through procurement mandate and market adoption.

Deployment: FMO — HACK core and COE co-packaged inside AI accelerator die. Decision Gate logic in firmware, Tier 1 hard stop in hardware logic. Photonic interface for inter-node attestation. Open standard, any manufacturer.

Key activities: FMO reference design published under CERN OHL. First HACK-compliant AI accelerator released by founding manufacturer. Government procurement mandates requiring HACK compliance adopted by member states. Second and third chip manufacturers implement HACK standard. Full global NAIA network operational.

Outputs: Universal silicon-level AI governance enforcement. Any AI system running on HACK-compliant silicon is governed regardless of jurisdiction or operator. The governance floor is hardware, not policy.

10. Deployment Model

10.1 Stage 1 — Linux VM

⚠ TBD — PMO. Software HACK via NVIDIA OpenShell policy engine and Artemis event stream. Claude via Anthropic API (see §8.1.4 for an alternative sovereign configuration). Grafana operations dashboard. PostgreSQL append-only audit log. OS: Ubuntu Server 24.04 LTS. Confirmed via two independent builds — OVHcloud VPS and local VMware VM. Target: Founding sandbox pilot — location to be confirmed.

10.2 Stage 1.5 — The Judge Protocol (TJP) Linux Image

⚠ TBD — PMO. Purpose-built Linux distribution, all components pre-integrated. Deployable on any x86 hardware or hypervisor. Claude via Anthropic API. Published under CERN OHL. OS: Ubuntu Server 24.04 LTS. Build process — see §8.1.3.

10.3 Stage 2 — Air-Gapped Appliance

⚠ TBD — TMO. Judge Protocol Image on certified hardware. TMO motherboard-mounted ASIC installed (§4.3.3, §8.2.2) — HACK module and COE chip identity not yet finalized (CV32E40S RISC-V and OpenTitan Ibex RISC-V have come up as early ideas, not evaluated candidates). GAE running locally on-device — reference implementation is Claude (Anthropic), per Evolution Clause; on-device deployment spec itself remains unresolved (§8.2.6). Model weights sealed at initialization. Rule set cryptographically sealed. OS: Ubuntu Server 24.04 LTS (FIPS 140-3 via Ubuntu Pro) — see §8.2.5, consistent with the PMO reference OS validated across two independent sandbox builds (§8.1.2). Hardened production deployment. Open item — GAE provider landscape: the GAE provider landscape is actively consolidating — recent hardware-vendor acquisitions of model/tooling platforms (e.g. The Judge Protocol (TJP)— Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

NVIDIA's 2026 acquisition of Hugging Face) illustrate that today's reference implementation may not reflect tomorrow's available options, or may face new compatibility constraints depending on the deployment's underlying hardware stack. The Evolution Clause principle — GAE selected per deployment, not fixed to any single provider — is the mechanism that keeps the Judge Protocol resilient to this kind of shift.

10.4 Stage 3 — HACK Chip (FMO)

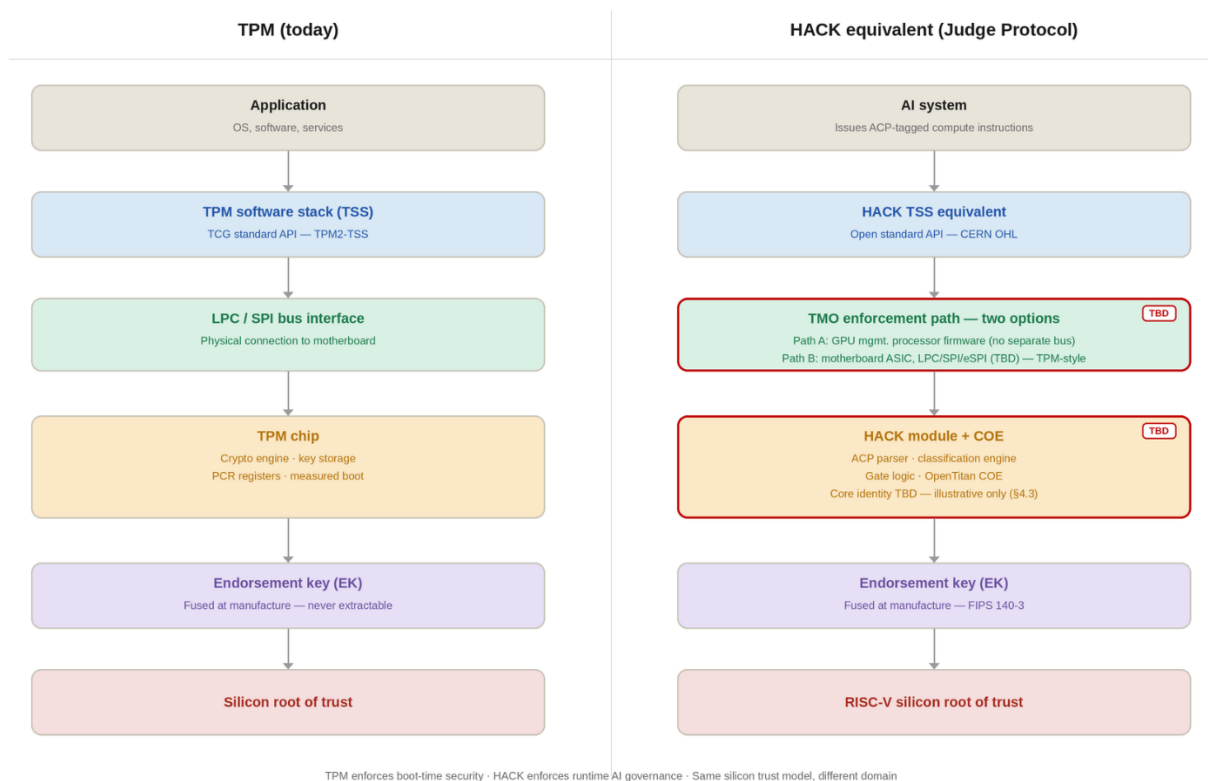
⚠ TBD — FMO. HACK core and COE co-packaged inside AI accelerator die, following the TPM precedent already established in §3.8 and Annex A.1 — a dedicated, tamper-resistant root-of-trust chip embedded directly in the compute substrate, the same architectural role TPM plays on today's motherboards. Specific core/ISA selection not yet finalized. Zephyr RTOS at firmware enforcement layer. Yocto minimal Linux on companion processor. Photonic inter-node attestation and ACK signalling. OS on host: Ubuntu Server LTS (EAL4+ certification target for government and IAEA-adjacent deployments). Open standard ratified as ISO/IEC.

Annex A — Visual Reference

This annex contains the reference diagrams for the Judge Protocol technical architecture. Both diagrams are referenced in Section 2 and throughout the document body.

A.1 TPM vs HACK Architecture

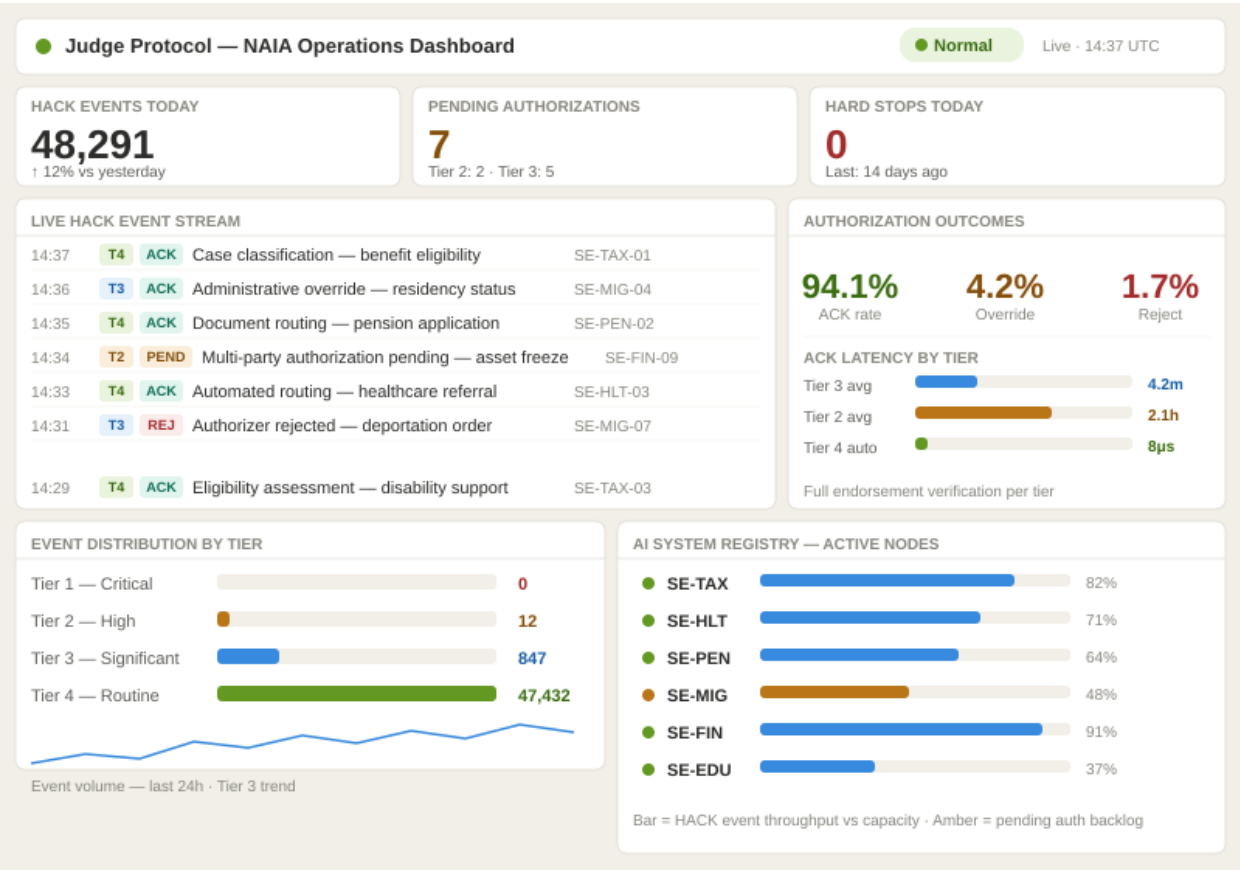
Comparative stack diagram: Trusted Platform Module architecture (left) alongside the HACK module equivalent (right). Each layer maps the proven TPM precedent to the Judge Protocol implementation.



A.2 NAIA Operations Dashboard

The Judge Protocol (TJP)— Technical Architecture HLD | Working Document v0.71 | September 2026 | Author: Alexander Hofmann | CC BY 4.0 / CERN OHL-S v2

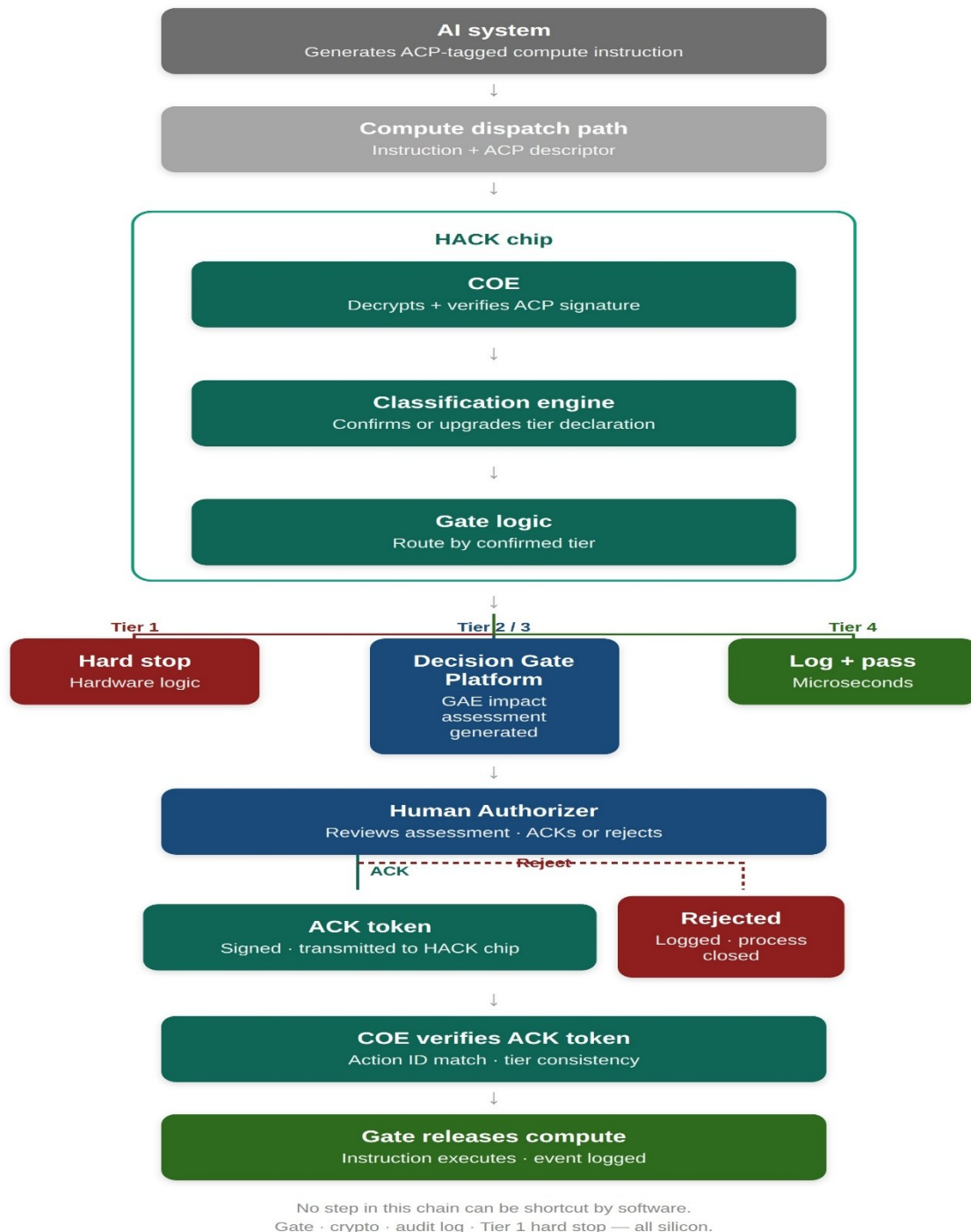
Reference mockup of the Grafana-based NAIA operations dashboard (Section 5.4). Shows the live HACK event stream, authorization outcomes, ACK latency by tier, event distribution by tier, and AI system registry node status. Built on Grafana consuming the Kafka event stream from the Decision Gate Platform.



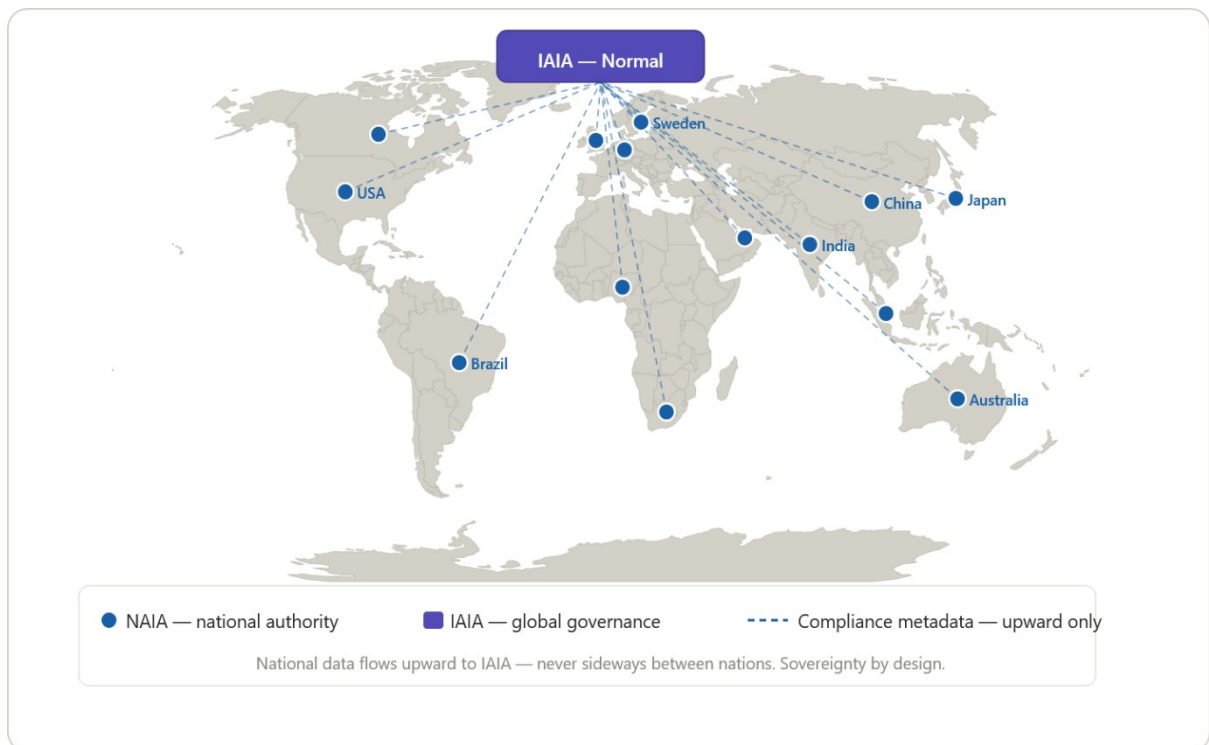
This document was developed in collaboration with Claude (Anthropic) as an AI research and drafting tool.

A.3 Hardware Action Flow

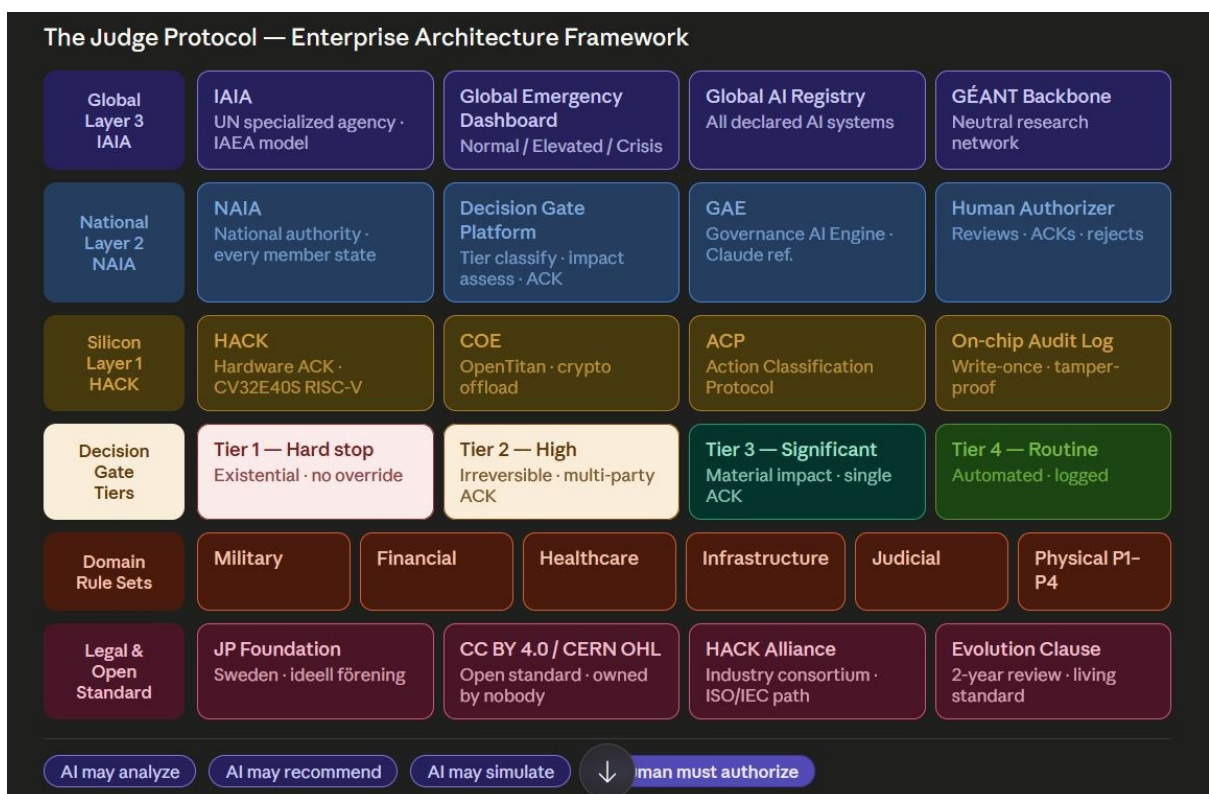
Step-by-step flowchart of a single AI compute instruction travelling from the AI system through the HACK enforcement chain to compute execution. Shows all four tier paths: Tier 1 hard stop in hardware logic; Tier 2 and 3 gate hold with human authorization; Tier 4 autonomous log and pass-through. Illustrates the complete enforcement chain described in Section 4.7 — no step can be shortcut by software. Gate, crypto, audit log and Tier 1 hard stop are all silicon.



A.4 — Global NAIA Map



A.5 — EA Framework



References Annex

Hardware cores:

CV32E40S — OpenHW Group, github.com/openhwgroup/cv32e40s
Ibex / OpenTitan — lowRISC, github.com/lowRISC/ibex + opentitan.org

Standards:

OSI model — ISO/IEC 7498-1
FIPS 140-3 — NIST, csrc.nist.gov
Ed25519 — RFC 8032
AES-256-GCM — NIST SP 800-38D
ChaCha20-Poly1305 — RFC 8439

Open source infrastructure:

GÉANT — geant.org
Open Policy Agent — openpolicyagent.org
Zephyr RTOS — zephyrproject.org
SUSE MicroOS — suse.com

Parent documents:

The Judge Protocol v0.95 — DOI: [10.5281/zenodo.20425548](https://doi.org/10.5281/zenodo.20425548)
Executive Summary v0.92 — DOI: [10.5281/zenodo.20401666](https://doi.org/10.5281/zenodo.20401666)